PROGRAMMING IN JAVA

ASSIGNMENT 3 – CO3

CO3 Coverage: Exception handling – exception hierarchy, checked and uncheckedexceptions, try, catch and finally, throw and throws, user-defined exceptions, exceptionchaining and propagation, try-with-resources and debugging; Multithreading – threadlifecycle, Thread and Runnable, thread methods, synchronization, deadlock, inter-threadcommunication, the Executor framework, Callable and Future, and concurrent collections(Sessions 27–38).

Mode: Pair Work – 2 Students per Group
Duration: 1 Week
Total Marks: 50
Submission Date: **12/11/2026**

Bloom's Levels Assessed: K4 – Analyse and K5 – Evaluate (the level of each task isstated under the task heading)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs. In this pair-work assignment, **bothstudents must execute every question in their own repository**, not only thepart for which they are primarily responsible.
- Create a GitHub repository named **CSA09-<RegisterNumber>-Assignment3**and share it with the course faculty (public, or private with the faculty added as acollaborator).
- Keep one folder for each task (for example: Task01, Task02 …or Q1, Q2 …), containing the Java source code, input files and output screenshots.
- Include a README.md with your name, register number, Java version, the stepsto compile and run each program, and a short note of the result of each task. • Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link (Student A): **192324156---RAKSHITH-R.R---CSA0928---PROGRAMMING-IN-JAVA/ASSIGNMENT - 3 at main · RAKS-rak57/192324156---RAKSHITH-R.R---CSA0928---PROGRAMMING-IN-JAVA**

GitHub Repository Link (Student B): **192324156---RAKSHITH-R.R---CSA0928---PROGRAMMING-IN-JAVA/ASSIGNMENT - 3 at main · RAKS-rak57/192324156---RAKSHITH-R.R---CSA0928---PROGRAMMING-IN-JAVA**

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODOwithworking code without changing the rest of the program, execute it and compare your output with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution wouldchange if the alternative concept were used, with code or pseudocode, and analyse andcompare the time complexity and space complexity of both solutions using Big-Onotation, supported by your own measurements wherever possible.
Pair-Work Principle

Both students are responsible for understanding the complete assignment. However, each question contains clearly identified individual responsibilities so that the contributionand understanding of each student can be independently verified.

Student A and Student B must not divide the assignment question-wise. Bothstudents must participate in every question.

The individual responsibilities intentionally alternate across questions so that bothstudents demonstrate

evidence of:

Design → Implementation → Analysis → Evaluation

Q1 – Handling Failure in a Bank ATM Simulation 6Marks**Bloom's Level:** K4 – Analyse

Problem Context – ATM Withdrawal

An ATM simulation reads the account number and withdrawal amount fromthe user, checks the balance and dispenses cash. Users may type letters instead of numbers, enter a negative amount, request more than the balance, or the card-reader file may bemissing.

Shared Group Work

Together:

- Identify every point at which the withdrawal can fail
- Classify each failure as an Error, a checked exception or an unchecked exception• Implement the withdrawal using try, catch (including multi-catch) and finally • Test the program with valid and invalid inputs

Student A – Exception Design

Student A must individually:

- Draw the part of the Throwable hierarchy relevant to the ATM • Prepare a table: failure point → exception type → checked/unchecked →whereit is caught
- Justify the order of the catch blocks
- Explain what the finally block guarantees in this program

Student B – Testing and Tracing

Student B must individually:

- Design at least five test inputs, each intended to trigger a different path • Record which catch block and whether the finally block executed for each input • Show the stack trace for one uncaught exception and explain each line • Demonstrate what happens if a broader catch block is placed before a narrower one

Joint Design Question

> *Should the ATM program catch every exception with a single catch (Exception e)? Under what conditions would that be acceptable, and what is lost by doing so?*

Both students must be able to defend the answer.

Alternative Concept and Complexity Analysis

If the ATM program avoided exceptions altogether – checking every input in advancewith Scanner.hasNextInt() and returning error codes such as -1 – what would the solutionbe? Compare readability and the number of checks needed, and analyse the time andspace cost of creating and throwing an exception versus returning an error code.

Q2 – User-Defined Exceptions and Propagation in an Online ExaminationPortal 7Marks

Bloom's Level: K4 – Analyse

Problem Context – Online Examination Portal

An online examination portal must reject a submission if the exam window has closed, if the student is not registered for the course, or if the answer file exceeds the size limit. These rules are checked in different layers: the controller, the service and the storagelayer.

Shared Group Work

Together:

- Design a hierarchy of user-defined exceptions for the portal (for example: ExamException with subclasses)

• Decide which exceptions are checked and which are unchecked • Implement throw and throws across the three layers

• Implement exception chaining so that the original cause is not lost

Student A – Exception Hierarchy and Propagation Design Student A must individually:

• Represent the exception hierarchy as a class diagram • Explain the reason for making each exception checked or unchecked • Draw the propagation path of one exception from the storage layer to the controller

• Explain where the exception is finally handled and what message the student sees

Student B – Failure Scenarios and Chaining Evidence

Student B must individually:

• Construct one scenario for each exception

• Demonstrate exception chaining using getCause() and show the full chained stack trace

• Show what information is lost if the exception is re-thrown without its cause • Record the observed behaviour in a test table

Joint Conclusion

Why is it better for the storage layer to throw a meaningful SubmissionStorageException than to let an IOException travel unchanged to the user interface?

Complete the Code

Complete TODO 1 below and execute the program.

```
import java.io.IOException;

class ExamException extends Exception {
    ExamException(String message) { super(message); }
    ExamException(String message, Throwable cause) { super(message, cause); }}

class SubmissionStorageException extends ExamException {
    SubmissionStorageException(String message, Throwable cause) { super(message,
    cause);
    }
}

class Storage {
    void save(String fileName, long sizeKb) throws SubmissionStorageException {try {
                if (sizeKb > 500) throw new IOException("Disk quota exceeded");
            System.out.println("Saved " + fileName);
        } catch (IOException e) {
            // TODO 1: Wrap the IOException in a SubmissionStorageException, keeping//
            IOException as its cause


        }
    }
}

public class StorageDemo {
    public static void main(String[] args) {
        Storage storage = new Storage();
        try {
            storage.save("answer01.pdf", 120);
            storage.save("answer02.pdf", 800);
        } catch (ExamException e) {
                    System.out.println("Error: " + e.getMessage());
            System.out.println("Cause: " + e.getCause().getMessage()); }
    }
}
```

Expected output:

```
Saved answer01.pdf
Error: Could not store answer02.pdf
Cause: Disk quota exceeded
```

Explain what would be printed in the Cause line if the IOException were not passedtothe constructor.

Alternative Concept and Complexity Analysis

Your custom exceptions are checked. If they were made unchecked (extendingRuntimeException), what would change in the method signatures and in the calling code?Analyse the time and space cost of capturing a stack trace for each exception and howchaining affects the memory held by one exception object.

Q3 – Resource Safety, Debugging and Robust Code 7MarksBloom's Level: K4 – Analyse

Here the responsibilities are deliberately reversed so that the same student does not always perform the design component.

Problem Context – Daily Sales Report Generator

A retail chain generates a daily sales report by reading several store files and writingasummary file. Occasionally the program stops with "Too many open files", and the report is sometimes incomplete.

Shared Group Work

Together:

- Implement the report generator first using try–catch–finally with manual close() • Re-implement it using try-with-resources
- Verify that both versions produce the same report

Student A – Debugging and Experimental Analysis

Student A must individually:

- Use breakpoints and step-through debugging to locate the point at which the manual version leaks a file
- Construct an input that makes the manual version fail part-way • Count the files left open in each version for the same failing input • Add logging that records which file caused the failure

Student B – Design and Correctness

Student B must individually:

- Identify every resource opened by the program and where it must be closed • Explain why the manual version can leak a resource when an exception occurs • Explain how try-with-resources guarantees closing, including the order of closing• Explain what a suppressed exception is and when it occurs

Joint Engineering Decision

Under what conditions is try-with-resources not sufficient on its own to make the report generator robust?

Alternative Concept and Complexity Analysis

If try-with-resources were replaced by explicit close() calls in a finally block, or by noclosing at all, what would the solution be? For a report that reads f store files, analysethetime complexity of processing and the maximum number of file handles (space) heldopen when a failure occurs in each version.

Q4 – Thread Creation and the Thread Life Cycle 7MarksBloom's Level: K4 – Analyse

Problem Context – Parallel Answer-Script Scanning

An examination cell scans answer scripts from four scanner stations at the same time. Each station is simulated as a thread that processes a batch of scripts and reports thenumber scanned.

Shared Group Work

Together:

- Implement the stations once by extending Thread and once by implementing Runnable
- Use start(), sleep(), join() and isAlive()
- Run the program several times and record the output order

Student A – Thread Design

Student A must individually:

- Explain the difference between the two ways of creating a thread and when eachis preferred
- Draw the thread life-cycle state diagram and mark the states each station passesthrough
- Explain why calling run() instead of start() does not create a new thread, with evidence
- Explain how join() ensures the final total is printed only after all stations finish

Student B – Behavioural Experiments

Student B must individually:

- Run the program at least ten times and record the order in which stations finish• Experiment with thread priorities and record whether the order changes • Convert one station into a daemon thread and record what happens when main() ends
- Explain the observations using the behaviour of the thread scheduler

Joint Conclusion

Why can the same multithreaded program produce different output orders on different runs, and why must a correct program not depend on that order?

Alternative Concept and Complexity Analysis

If the four scanner stations were run using an ExecutorService with a fixed thread pool instead of creating Thread objects yourself, what would the solution be? For n scripts andp threads, analyse the expected completion time (about n/p batches) and the memoryused per thread (thread stack) in each approach.

Q5 – Race Conditions in a Cinema Seat-Booking System 6Marks**Bloom's Level:** K4 – Analyse

Problem Context – Cinema Seat Booking

A cinema app allows many users to book seats at the same time. Each booking checkswhether a seat is free and then marks it as booked. Customers have complained that thesame seat was sold twice.

Shared Group Work

Together:

- Implement the booking without synchronization and run it with many threads • Implement a synchronized method version and a synchronized block version • Verify that the synchronized versions never double-book

Student A – Producing and Explaining the Race

Student A must individually:

- Design a test that reliably reproduces double booking
- Record how many double bookings occur over repeated runs • Explain the race condition step by step using an interleaving of two threads • Identify the exact critical section

Student B – Comparing Synchronization Strategies

Student B must individually:

- Measure the time to process 10,000 bookings with each version • Compare locking the whole method with locking only the critical section • Explain static synchronization and when it would be needed here • Explain the intrinsic lock that each version uses

Joint Engineering Decision

Which synchronization strategy would you deploy for a cinema with 20 screens, and why?

Complete the Code

Run the program first with TODO 1 written WITHOUT the synchronized keyword andrecord the result over 10 runs. Then complete TODO 1 correctly and execute it again.

```
public class SeatBooking {
    private final boolean[] booked;
    private int bookingsMade = 0;

    SeatBooking(int seats) { booked = new boolean[seats]; }

    // TODO 1: Make the check-then-book operation atomic so that a seat can never be// sold


    public static void main(String[] args) throws InterruptedException { SeatBooking
        screen = new SeatBooking(10);
        Thread[] users = new Thread[100];
        for (int i = 0; i < users.length; i++) {
            final int seat = i % 10; // 10 users compete for everyusers[i] = new Thread(()
            screen.book(seat));
            users[i].start();
        }
        for (Thread t : users) t.join();
        System.out.println("Seats sold: " + screen.bookingsMade); }
}
```

seat

Expected output:

```
Seats sold: 10
```

Explain why the unsynchronized version may sell more than 10 seats. **Alternative**

Concept and Complexity Analysis

If the synchronized method were replaced by an AtomicBoolean[] usingcompareAndSet(), a ReentrantLock, or a ConcurrentHashMap with putIfAbsent(), what would the booking solution be? Analyse the time per booking under contention andtheextra space each approach uses for s seats.

Q6 – Deadlock and Inter-Thread Communication 7Marks**Bloom's Level:** K4 – Analyse

Problem Context – Hostel Wallet Transfers and Mess Token Counter

Part A: Students transfer money between hostel wallet accounts; each transfer locks bothaccounts. Part B: A mess counter prepares meal tokens (producer) and students collect them (consumers) from a counter that can hold at most 5 tokens.

Shared Group Work

Together:

- Part A: implement transfers that can deadlock, then a version that cannot • Part B: implement the producer–consumer counter using wait(), notify() and notifyAll()
- Test both parts with several threads

Student A – Producer–Consumer Design

Student A must individually:

- Explain why wait() must be called inside a loop and inside a synchronized block • Show what goes wrong when notify() is used instead of notifyAll() with several consumers, or explain why it does not in your design

• Record the counter size over time to show it never exceeds 5 or drops below0• Explain the difference between sleep() and wait()

Student B – Deadlock Analysis and Prevention

Student B must individually:

• Construct a pair of transfers that reliably deadlocks

• Identify the four conditions for deadlock present in the program • Prevent the deadlock using a consistent lock-ordering rule • Explain why the rule works

Joint Conclusion

What do deadlock and the producer–consumer problem have in common, and how does each design ensure that threads make progress?

Complete the Code

Complete TODO 1 and TODO 2 below and execute the program.

```
public class TokenCounter {
    private final int[] buffer = new int[5]; // the counter holds at most 5 tokensprivate i
    count = 0, in = 0, out = 0;

    public synchronized void put(int token) throws InterruptedException { // TODO 1: Wait wh
        the counter is full, add the token at position in, then// wake waiting threads

    }

    public synchronized int take() throws InterruptedException { // TODO 2: Wait while the
        counter is empty, remove the token at position out,// wake waiting threads and retu
        the token

    }

    public static void main(String[] args) throws InterruptedException { TokenCounter
        counter = new TokenCounter();
        int[] collected = new int[2];
        Thread cook = new Thread(() -> {
            try {
                                for (int t = 1; t <= 20; t++) counter.put(t);
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }});
        Thread[] students = new Thread[2];
        for (int s = 0; s < 2; s++) {
            final int id = s;
            students[s] = new Thread(() -> {
                try {
                     for (int k = 0; k < 10; k++) collected[id] += counter.take();
             } catch (InterruptedException e) { Thread.currentThread().interrupt();
            });
        }
        cook.start(); for (Thread t : students) t.start();
        cook.join(); for (Thread t : students) t.join();
        System.out.println("Sum of tokens collected: " + (collected[0] + collected[1])+
                " (expected 210)");
    }
}
```

}

Expected output:

```
Sum of tokens collected: 210 (expected 210)
```

Change while to if in both methods, run it several times, and explain any difference. **Alternative**

Concept and Complexity Analysis

If wait()/notifyAll() were replaced by an ArrayBlockingQueue<Integer> of capacity 5, andlock ordering were replaced by tryLock() with a timeout, what would the solutions be?Analyse the time per put/take and per transfer, and the space used, and state whichdesign is less error-prone.

Q7 – Design a Concurrent, Fault-Tolerant Processor from First Principles 10MarksBloom's

Level: K5 – Evaluate

Problem Context – Semester Result Processing

The examination cell must process 10,000 answer-script score files at the end of thesemester. Each file must be read, validated and used to compute the student's result. Some files are corrupt or missing. The processing must finish quickly, a single badfilemust not stop the whole run, and a summary of successes and failures must be producedat the end.
You are not given the design. You must develop it from the requirements. **Stage 1 – Interpret**

the Problem (Joint) | 2 Marks

• Identify the tasks that can run in parallel and the data that is shared • Identify every failure that can occur and how each must be handled • State the requirements as measurable statements

Stage 2 – Design (Student A) | 3 Marks

• Design the solution using ExecutorService, Callable and Future • Choose a concurrent collection for the shared results and justify it • Design the exception handling so that one failure does not stop the run • Present the design as a class diagram and pseudocode

Stage 3 – Implement and Measure (Student B) | 3 Marks • Implement the design and inject corrupt and missing files • Measure the total time with thread pools of 1, 2, 4 and 8 threads • Verify that the summary counts are correct for a small hand-checked set • Record any failure that was not handled correctly

Stage 4 – Defend the Design (Joint) | 2 Marks

• Explain the pool size you would recommend and why more threads did not always help
• Explain how your design guarantees a complete and correct summary • Identify one remaining limitation

Alternative Concept and Complexity Analysis

If the 10,000 files were processed using parallel streams (files.parallelStream()) or bycreating one thread per file, instead of an ExecutorService, what would the solution be?Analyse the total time and the memory used (number of threads × stack size) in eachapproach, and how a failure in one file is handled.

MARK DISTRIBUTION

Question	Assessed Area	Bloom's Level	Marks
Q1	Exception hierarchy, try, catch and t	K4 –	Analys
Q2	User-defined exceptions, propaga	cha	Analys
Q3	try-with-resources and debugging		Analys
Q4	Thread creation and life cycle		Analys
Q5	Race conditions and synchronization		Analys
Q6	Deadlock and inter-thread communication		Analys
Q7	Concurrent, fault-tolerant design fromprinciples	first K5 –	Evaluat
	TOTAL		**50**

Individual Contribution Record

Complete the following table and submit it with the report:

Question	Student A Responsibility	Student B – PrimaryRespons	Joint
Q1			
Q2			
Q3			
Q4			
Q5			
Q6			
Q7			

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, with input files, output screenshots and a README.md.

- One joint report with each individual section clearly labelled Student A or Student B
- Java source code for all questions
- Test tables, stack traces and logs used as evidence
- Raw measurement data with tables/graphs

Assessment Verification

Each student will be verified individually. Either student may be asked to explain any part of the assignment, including the part for which the other student was primarilyresponsible.

SIMATS ENGINEERING

CSA0928 – Programming in Java

ASSIGNMENT 3 – REPORT

Exception Handling and Multithreading (CO3) · Pair work · 50 marks

Item	Details Student A	Details Student B
Student name	**RAKSHITH R.R**	**RAKSHITH R.R**
Register number	**192324156**	**192324156**
Department / Section	**B.Tech -AI&DS**	**B.Tech -AI&DS**
Java version used	**OpenJDK 21**	**OpenJDK 21**
GitHub repository link	**192324156---RAKSHITH-R.R---CSA0928---PROGRAMMING-IN-JAVA/ASSIGNMENT - 3 at main · RAKS-rak57/192324156---RAKSHITH-R.R---CSA0928---PROGRAMMING-IN-JAVA**	**192324156---RAKSHITH-R.R---CSA0928---PROGRAMMING-IN-JAVA/ASSIGNMENT - 3 at main · RAKS-rak57/192324156---RAKSHITH-R.R---CSA0928---PROGRAMMING-IN-JAVA**
Submission date	**12/11/2026**	**12/11/2026**

Placeholders in [ ] must be filled in before submission. The assignment document is reproduced on the preceding pages; the answers follow. Each individual section is labelled Student A or Student B; both students executed every program.

Individual Contribution Record

The primary responsibilities below are exactly those defined in the assignment for each question. Both students ran, tested and can defend every question.

Question	Student A – primary responsibility	Student B – primary responsibility	Joint
Q1	Exception design: Throwable hierarchy, failure → exception table, catch order, role of finally	Testing and tracing: test inputs, catch/finally record, stack trace of an uncaught exception, broad-before-narrow demonstration	Failure points, classification, implementation, testing; the catch (Exception) question; error-code alternative
Q2	Class diagram of the exception hierarchy, checked/unchecked reasons, propagation path, final handling and message	One scenario per exception, chaining with getCause(), full chained trace, what is lost without the cause, test table	Hierarchy, throw/throws across three layers, chaining; conclusion; StorageDemo
Q3	Debugging (breakpoints, step-through), failing input, open-file count per version, logging	Resources and where they must be closed, why manual leaks, try-with-resources guarantee and order, suppressed exceptions	Manual and try-with-resources versions, same report; engineering decision; alternative analysis
Q4	Thread vs Runnable, life-cycle diagram, run() vs start() evidence, join() explanation	12+ runs, priorities, daemon thread, scheduler explanation	Both implementations; conclusion; executor analysis
Q5	Reliable test for double booking, count over repeated runs, step-by-step interleaving, critical section	Timing of 10,000 bookings, lock scope, static synchronization, intrinsic locks	Three versions; decision for 20 screens; SeatBooking; alternatives
Q6	Producer–consumer: wait() in a loop, notify() vs notifyAll(), counter size trace, sleep() vs wait()	Reliable deadlock, four conditions, lock-ordering prevention and why it works	Both parts; TokenCounter; conclusion; ArrayBlockingQueue/tryLock analysis
Q7	Stage 2 – design: executor/Callable/Future,	Stage 3 – implementation, fault injection, timing with	Stage 1 – interpretation; Stage 4 – defence; alternative analysis

	concurrent collection, exception design, class diagram, pseudocode	1/2/4/8 threads, hand-checked set, unhandled failure	

Execution Environment and How the Measurements Were Taken

Item	Value
Java	OpenJDK 21.0.12 (HotSpot), compiled with javac -g
Machine used for the recorded outputs	Linux container, **1 CPU core** (availableProcessors() = 1), default thread stack 1,024 KB, default ulimit -n 20,000
Timing method	System.nanoTime(); warm-up runs first (JIT); best or median of repeated runs as stated in each table
Raw data	Every program's output is saved in Qn/output/*.txt (and as screenshots); Q7/output/measurements.csv holds every timed Q7 run; charts are in docs/charts
Important limitation	With one core, CPU-bound programs cannot run in parallel, so speed-ups from threads appear only when threads **wait** (sleep or I/O). Race conditions that need true parallelism were reproduced by widening the window with Thread.yield(), and this is stated wherever it was done. **All timing experiments should be re-run on a multi-core computer (./run_all.sh) and the tables compared; the conclusions about lock scope, space and complexity do not depend on the number of cores.**

Summary of results

Q	Marks	Headline result
1	6	12/12 ATM tests; finally ran on all 12 paths; exception ≈ 130× slower than an error code per failing input but equal on success
2	7	StorageDemo matches the expected output; 8/8 portal scenarios; chaining doubles trace time/memory (≈ 2×)
3	7	Leaky version leaves 2 handles open and a 0-byte report; manual finally and try-with-resources leave 0; "Too many open files" reproduced

4	7	24 recorded runs, different order each time, total always 80; pool follows the n/p model (efficiency 0.97 at p = 8)
5	6	Seats sold: 10; natural race not seen in 10 runs, 2,498/2,500 double bookings with the window widened; block lock 8× faster than method lock with a 100 µs payment step
6	7	Sum … 210; if gave a wrong sum in 8/10 runs; notify() hung 10/10 with several producers; deadlock detected and prevented by lock ordering
7	10	Summary correct for 15 hand-checked and 10,000 generated files at every pool size; defect found and fixed; near-linear speed-up (31.8× at 32 threads) for latency-bound work

Q1 – Handling Failure in a Bank ATM Simulation (6 marks, K4)

Shared group work – failure points, classification, implementation, testing

Every point at which a withdrawal can fail was identified by walking through the steps of the withdrawal (read account → read amount → validate → open card-reader file → find account → compare balance → dispense). The classification follows the Java rule: an **Error** signals a JVM problem the program cannot repair; a **checked exception** is an expected, recoverable condition that the compiler forces the caller to handle or declare; an **unchecked exception** (RuntimeException) is a defect or invalid argument that the caller could have avoided.

```
import java.io.BufferedReader;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;
import java.util.InputMismatchException;
import java.util.NoSuchElementException;
import java.util.Scanner;

/** Q1 - ATM withdrawal with try / catch (incl. multi-catch) / finally. */
public class Atm {
   static int finallyRuns = 0;                    // how many times a finally block has executed

   /** Returns the PATH taken (SUCCESS or the name of the failure) so that tests can check it. */
   static String withdraw(String request, String cardFile) {
      BufferedReader reader = null;
      String path = "SUCCESS";
      try {
         Scanner in = new Scanner(request);
         int account = in.nextInt();              // letters      -> InputMismatchException (unchecked)
         int amount = in.nextInt();               // missing token-> NoSuchElementException (unchecked)
         if (amount <= 0)                         // negative / zero -> our own unchecked failure
            throw new IllegalArgumentException("Amount must be positive: " + amount);
         reader = new BufferedReader(new FileReader(cardFile));      // missing file -> FileNotFoundException (checked)
         int balance = findBalance(reader, account);                 // IOException, AccountNotFound, CardData (checked)
         if (amount > balance) throw new InsufficientFundsException(balance, amount);
         System.out.println("   dispensed " + amount + ", new balance " + (balance - amount));

      } catch (InputMismatchException | IllegalArgumentException e) {    // MULTI-CATCH: two unrelated types, same
handling
```

```
            path = "BAD_INPUT";
            System.out.println("   Please enter a valid account number and a positive amount (" + e.getClass().getSimpleName() +
")");
        } catch (NoSuchElementException e) {          // superclass of InputMismatchException: must come AFTER it
            path = "MISSING_INPUT";
            System.out.println("   Please enter both the account number and the amount");
        } catch (InsufficientFundsException e) {
            path = "INSUFFICIENT_FUNDS";
            System.out.println("   " + e.getMessage() + " (short by " + e.getShortfall() + ")");
        } catch (AccountNotFoundException e) {
            path = "ACCOUNT_NOT_FOUND";
            System.out.println("   " + e.getMessage());
        } catch (CardDataException e) {
            path = "CARD_DATA";
            System.out.println("   Card data damaged: " + e.getMessage() + "  [cause: " + e.getCause().getClass().getSimpleName()
+ "]");
        } catch (FileNotFoundException e) {           // subclass of IOException: must come BEFORE it
            path = "CARD_FILE_MISSING";
            System.out.println("   Card reader unavailable: " + cardFile);
        } catch (IOException e) {
            path = "IO_ERROR";
            System.out.println("   Card reader error: " + e.getMessage());
        } catch (Exception e) {                       // last-resort safety net, AFTER every specific handler
            path = "UNEXPECTED";
            System.out.println("   Unexpected problem, transaction cancelled: " + e);
        } finally {
            finallyRuns++;                            // runs on EVERY path: success, handled failure, even return
            System.out.println("   [finally] card reader released, session closed");
            if (reader != null) {
                try { reader.close(); } catch (IOException ignored) { }
            }
        }
        return path;
    }

    private static int findBalance(BufferedReader r, int account)
            throws IOException, AccountNotFoundException, CardDataException {
        String line;
        while ((line = r.readLine()) != null) {
            String[] p = line.split(",");
```

```
            try {
                if (Integer.parseInt(p[0].trim()) == account) return Integer.parseInt(p[1].trim());
            } catch (NumberFormatException | ArrayIndexOutOfBoundsException e) {
                throw new CardDataException("bad line \"" + line + "\"", e);   // wrap: file damage is not the user's mistake
            }
        }
        throw new AccountNotFoundException(account);
    }
}
```

Student A – Exception design

A1. Part of the Throwable hierarchy used by the ATM

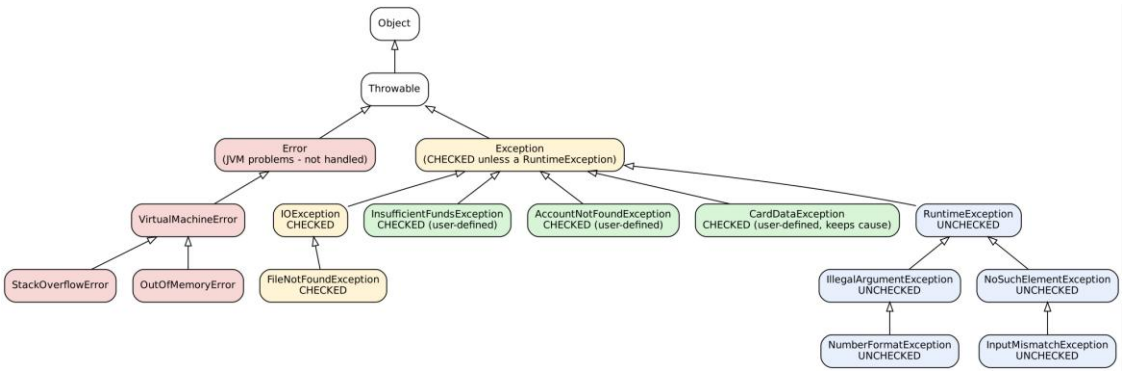


Figure Q1.1 – Throwable hierarchy relevant to the ATM (red = Error, yellow = checked, blue = unchecked, green = user-defined checked). The superclass chains were also printed by ErrorDemo (Q1/output).

A2. Failure point → exception type → checked / unchecked → where it is caught

#	Failure point	Exception type	Checked?	Caught in (Atm.withdraw)
1	Letters typed for the account number or amount (nextInt())	InputMismatchException	Unchecked	catch 1 – multi-catch InputMismatchException | IllegalArgumentException → BAD_INPUT
2	Negative or zero amount	IllegalArgumentException (thrown by the program)	Unchecked	catch 1 – same multi-catch
3	Amount missing (only one token typed)	NoSuchElementException	Unchecked	catch 2 → MISSING_INPUT

4	More than the balance	InsufficientFundsException (user-defined)	Checked	catch 3 → INSUFFICIENT_FUNDS
5	Account number not in the card file	AccountNotFoundException (user-defined)	Checked	catch 4 → ACCOUNT_NOT_FOUND
6	Card-file line damaged (Integer.parseInt fails)	NumberFormatException wrapped in CardDataException (cause kept)	Checked (wrapper)	catch 5 → CARD_DATA
7	Card-reader file missing (or a directory)	FileNotFoundException	Checked	catch 6 → CARD_FILE_MISSING
8	Read error while reading the card file	IOException	Checked	catch 7 → IO_ERROR
9	A bug in the ATM code (e.g. null reference)	RuntimeException subclasses	Unchecked	catch 8 Exception – last resort → UNEXPECTED
10	JVM out of memory / stack overflow while dispensing	OutOfMemoryError, StackOverflowError	**Error**	**Not caught.** ErrorDemo shows catch (Exception) does not catch a StackOverflowError; only the JVM's top level reports it.

Row 6 is a design decision: NumberFormatException is *unchecked*, but when it is raised while reading the **card file** it is not the customer's mistake. If it escaped unchanged it would be caught by the multi-catch of row 1 and the customer would be told "enter a valid amount" for a damaged card. It is therefore **wrapped** in the checked CardDataException, keeping the original as the cause (see the CARD_DATA test below).

A3. Justification of the order of the catch blocks

Java tests the catch blocks from top to bottom and runs the first one whose type matches, so a **subclass must appear before its superclass** (the compiler rejects the opposite order – see Student B). The order used:

1. InputMismatchException | IllegalArgumentException – the user-input failures; unrelated types, so a multi-catch is legal and avoids duplicating the handler.
2. NoSuchElementException – **superclass** of InputMismatchException; it had to come after catch 1, otherwise catch 1's InputMismatchException would be unreachable.

3. InsufficientFundsException, AccountNotFoundException, CardDataException – unrelated business failures, each with its own message.
4. FileNotFoundException – **subclass** of IOException, so it precedes IOException.
5. IOException – all remaining I/O failures.
6. Exception – last resort only, after every specific handler.

A4. What finally guarantees in this program

The finally block runs **on every path out of the try**: normal completion, any handled exception, and even a return or an exception thrown from inside a catch block. In the ATM it increments finallyRuns, prints "card reader released, session closed", and closes the BufferedReader (inside its own try/catch because close() can throw). The test table below shows it executed for all 12 inputs. What it does **not** guarantee: it will not run if the JVM is terminated (System.exit, power loss) or the thread is killed, and an exception thrown *from* the finally block would replace the original one.

Student B – Testing and tracing

B1. Test inputs – the catch block taken and whether finally ran

AtmTest reads Q1/input/tests.txt and checks both the path and that finally executed exactly once per call.

#	Test	Request	Card file	Expected path (catch)	Actual path	finally ran	Result
1	valid withdrawal	1001 500	cards.txt	SUCCESS	SUCCESS	yes	PASS
2	exact balance (boundary)	1002 200	cards.txt	SUCCESS	SUCCESS	yes	PASS
3	letters instead of number	abc 500	cards.txt	BAD_INPUT	BAD_INPUT	yes	PASS
4	letters in the amount	1001 five	cards.txt	BAD_INPUT	BAD_INPUT	yes	PASS
5	negative amount	1001 -50	cards.txt	BAD_INPUT	BAD_INPUT	yes	PASS
6	zero amount	1001 0	cards.txt	BAD_INPUT	BAD_INPUT	yes	PASS

7	amount token missing	1001	cards.txt	MISSING_INPUT	MISSING_INPUT	yes	PASS
8	more than the balance	1002 900	cards.txt	INSUFFICIENT_FUNDS	INSUFFICIENT_FUNDS	yes	PASS
9	unknown account	9999 100	cards.txt	ACCOUNT_NOT_FOUND	ACCOUNT_NOT_FOUND	yes	PASS
10	card file missing	1001 100	no_such_file.txt	CARD_FILE_MISSING	CARD_FILE_MISSING	yes	PASS
11	card file damaged	1002 100	cards_corrupt.txt	CARD_DATA	CARD_DATA	yes	PASS
12	card path is a directory	1001 100	input	CARD_FILE_MISSING	CARD_FILE_MISSING	yes	PASS

Result: **12 / 12** tests passed and **12 finally blocks executed** (Q1/output/AtmTest.txt). **A correction recorded honestly:** my first expectation for test 12 (path that is a directory) was IO_ERROR. The run showed CARD_FILE_MISSING – FileReader reports a directory as FileNotFoundException ("Is a directory"). The expected value in tests.txt was corrected; the program was right.

```
Q1 - AtmTest

$ java -cp out AtmTest
Test 1: valid withdrawal   request="1001 500" card=input/cards.txt
   dispensed 500, new balance 4500
   [finally] card reader released, session closed
   -> path=SUCCESS expected=SUCCESS finally executed=true  PASS
Test 2: exact balance (boundary)   request="1002 200" card=input/cards.txt
   dispensed 200, new balance 0
   [finally] card reader released, session closed
   -> path=SUCCESS expected=SUCCESS finally executed=true  PASS
Test 3: letters instead of number   request="abc 500" card=input/cards.txt
   Please enter a valid account number and a positive amount (InputMismatchException)
   [finally] card reader released, session closed
   -> path=BAD_INPUT expected=BAD_INPUT finally executed=true  PASS
Test 4: letters in the amount   request="1001 five" card=input/cards.txt
   Please enter a valid account number and a positive amount (InputMismatchException)
   [finally] card reader released, session closed
   -> path=BAD_INPUT expected=BAD_INPUT finally executed=true  PASS
Test 5: negative amount   request="1001 -50" card=input/cards.txt
   Please enter a valid account number and a positive amount (IllegalArgumentException)
   [finally] card reader released, session closed
   -> path=BAD_INPUT expected=BAD_INPUT finally executed=true  PASS
Test 6: zero amount   request="1001 0" card=input/cards.txt
   Please enter a valid account number and a positive amount (IllegalArgumentException)
   [finally] card reader released, session closed
   -> path=BAD_INPUT expected=BAD_INPUT finally executed=true  PASS
Test 7: amount token missing   request="1001" card=input/cards.txt
   Please enter both the account number and the amount
   [finally] card reader released, session closed
   -> path=MISSING_INPUT expected=MISSING_INPUT finally executed=true  PASS
Test 8: more than the balance   request="1002 900" card=input/cards.txt
   Insufficient funds: balance 200, requested 900 (short by 700)
   [finally] card reader released, session closed
   -> path=INSUFFICIENT_FUNDS expected=INSUFFICIENT_FUNDS finally executed=true  PASS
Test 9: unknown account   request="9999 100" card=input/cards.txt
   Account not found: 9999
   [finally] card reader released, session closed
   -> path=ACCOUNT_NOT_FOUND expected=ACCOUNT_NOT_FOUND finally executed=true  PASS
Test 10: card file missing   request="1001 100" card=input/no_such_file.txt
   Card reader unavailable: input/no_such_file.txt
   [finally] card reader released, session closed
   -> path=CARD_FILE_MISSING expected=CARD_FILE_MISSING finally executed=true  PASS
Test 11: card file damaged   request="1002 100" card=input/cards_corrupt.txt
   Card data damaged: bad line "1002,abc"  [cause: NumberFormatException]
   [finally] card reader released, session closed
   -> path=CARD_DATA expected=CARD_DATA finally executed=true  PASS
Test 12: card path is a directory   request="1001 100" card=input
   Card reader unavailable: input
   [finally] card reader released, session closed
   -> path=CARD_FILE_MISSING expected=CARD_FILE_MISSING finally executed=true  PASS

Result: 12 / 12 tests passed; finally blocks executed = 12
```

Figure Q1.2 – Output of AtmTest (first 50 lines)

B2. Stack trace of an uncaught exception, line by line

```
$ java -cp out UncaughtDemo 2>&1; echo "exit code = $?"
ATM started
Exception in thread "main" java.lang.NumberFormatException: For input string: "abc"
    at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
    at java.base/java.lang.Integer.parseInt(Integer.java:662)
    at java.base/java.lang.Integer.parseInt(Integer.java:778)
    at UncaughtDemo.parseAmount(UncaughtDemo.java:13)
```

```
    at UncaughtDemo.process(UncaughtDemo.java:9)
    at UncaughtDemo.main(UncaughtDemo.java:5)
exit code = 1
```

Line of the trace	**Meaning**
Exception in thread "main" java.lang.NumberFormatException: For input string: "abc"	Which thread died, the exception class and its message (the text that could not be converted).
at java.base/java.lang.NumberFormatException.forInputString(...)	Top of the stack = where the exception was **created** (inside the JDK).
at java.base/java.lang.Integer.parseInt(...) (two lines)	The JDK method that detected the bad text and threw.
at UncaughtDemo.parseAmount(UncaughtDemo.java:13)	**Our** code that called parseInt – the first line that belongs to the program (where to start debugging).
at UncaughtDemo.process(UncaughtDemo.java:9)	The caller of parseAmount; no try/catch, so the exception kept travelling up.
at UncaughtDemo.main(UncaughtDemo.java:5)	Bottom of the stack: where the program started. No handler anywhere, so the JVM printed the trace and ended the thread.
exit code = 1	The process ended abnormally; "this line is never printed" shows execution never returned to main.

B3. A broader catch block placed before a narrower one

```
import java.io.*;
public class BroadBeforeNarrow {
    public static void main(String[] a) {
        try { new FileReader("x.txt").close(); }
        catch (Exception e) { System.out.println("any exception"); }      // broad handler first
        catch (FileNotFoundException e) { System.out.println("file missing"); }  // can never be reached
```

```
    }
}
```

```
$ for f in negative/*.java; do echo "### javac $f"; javac -d /tmp/neg $f 2>&1; done
### javac negative/BroadBeforeNarrow.java
negative/BroadBeforeNarrow.java:6: error: exception FileNotFoundException has already been caught
        catch (FileNotFoundException e) { System.out.println("file missing"); }  // can never be reached
        ^
1 error
### javac negative/MultiCatchRelated.java
negative/MultiCatchRelated.java:4: error: Alternatives in a multi-catch statement cannot be related by subclassing
        catch (NumberFormatException | IllegalArgumentException e) { }       // NumberFormatException IS-A
IllegalArgumentException
                               ^
1 error
### javac negative/UnreportedChecked.java
negative/UnreportedChecked.java:4: error: unreported exception FileNotFoundException; must be caught or declared to
be thrown
        ^
```

The compiler refuses to compile it: exception FileNotFoundException has already been caught. The broad catch (Exception) would take every exception first, so the narrow handler could never run. The same output shows the other two related compile-time rules that the ATM relies on: a multi-catch whose alternatives are related by subclassing is rejected, and a checked exception that is neither caught nor declared is rejected.

Joint design question – should the ATM catch everything with one catch (Exception e)?

No, not as the only handler. It is acceptable only in two situations: (1) as the **last** handler at a program boundary (the catch (Exception) of withdraw), after every specific handler, so that an unforeseen bug cancels the transaction safely instead of crashing the ATM and the card is still released; (2) when the failure is logged with its stack trace and nothing else can sensibly be done. **What is lost:** the ability to give the customer the right message ("insufficient funds" and "card reader unavailable" need different responses and different recovery); the compiler's help (checked exceptions you forgot to think about disappear into the catch-all); the distinction between customer mistakes, system faults and bugs; and, if the handler is empty or only prints, the evidence needed to find a defect (a NullPointerException becomes indistinguishable from a bad amount). It also swallows InterruptedException and similar signals that should be propagated.

Alternative Concept and Complexity Analysis – no exceptions, error codes

What the solution would be. Every input is validated before use (Scanner.hasNextInt(), amount > 0, file exists, account found, balance sufficient) and each method returns a code (0 success, -1 bad input, -2 negative amount, -3 file missing, -4 account not found, -5 insufficient funds, -6 damaged data). The caller tests the code after every call.

```
int code = withdrawCode(request, cardFile);
if (code == -1) System.out.println("Enter valid numbers");
else if (code == -2) System.out.println("Amount must be positive");
else if (code == -3) System.out.println("Card reader unavailable");
else if (code == -4) ...  else if (code == -5) ...  else if (code == -6) ...  else dispense();


static int withdrawCode(String request, String file) {
   Scanner in = new Scanner(request);
   if (!in.hasNextInt()) return -1;  int account = in.nextInt();
   if (!in.hasNextInt()) return -1;  int amount  = in.nextInt();
   if (amount <= 0) return -2;
   if (!new java.io.File(file).exists()) return -3;       // still needs try/catch for IOException: the file can vanish after the check
   ...
```

Criterion	Exceptions (implemented)	Error codes
Readability	Normal path reads top to bottom; each failure has its own handler with a name	Every call is followed by an if (code == ...); the useful path is buried; magic numbers must be documented
Number of checks	One try + 8 catch blocks; the failing statement itself raises the problem – no pre-checks	At least 6 explicit tests before/after the calls (hasNextInt ×2, amount, file exists, account found, balance) **plus** the caller's if/else chain on the code **plus** a try/catch anyway for IOException, which Java still forces
Safety	A forgotten failure cannot be silently ignored (checked) or at least stops the program (unchecked)	A forgotten if (code ...) silently continues with bad data; a time-of-check/time-of-use gap (file deleted after exists())

Failure information	Type, message, stack trace, cause chain	One integer

Time and space cost (measured by ExceptionCostBenchmark, results in Q1/output and Figure Q1.3):

Share of invalid inputs	**try / catch (ns per input)**	**check first + error code (ns per input)**
0 %	4.4	5.9
1 %	13.0	16.3
10 %	50.5	15.6
50 %	200.4	15.1
100 %	362.5	2.8

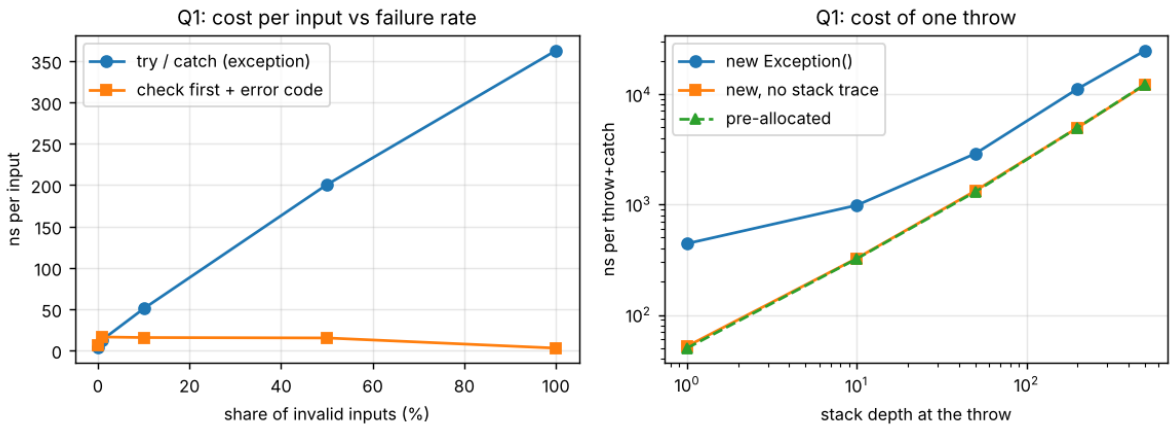


Figure Q1.3 – Left: cost per input vs share of invalid inputs. Right: cost of one throw by stack depth (log–log).

Analysis. On the **success path** an exception costs nothing extra (4.4 ns versus 5.9 ns for the extra pre-check – the check is even slightly slower). Each **failure** costs a lot: creating the exception fills in its stack trace, which walks the call stack, so the cost of one throw grows with the stack depth – Θ(d) (442 ns at depth 1, 24,676 ns at depth 500; the table in ExceptionCostBenchmark.txt). At 100 % invalid input the exception version is about 130× slower per input (362 ns versus 2.8 ns). Part of that cost is only the stack trace: an exception created without a stack trace is 3–8× cheaper, but it still grows with depth because unwinding the frames is also Θ(d). **Space:** an exception object holds its stack trace – about 724 bytes at depth 1, 2,796 bytes at depth 100, and 8,116 bytes once getStackTrace() has expanded it to StackTraceElement objects (Θ(d)); an error code is a 4-byte int, Θ(1). **Conclusion:** exceptions are the right tool for *exceptional* conditions; at the ATM's rate (a few invalid entries per minute) the microseconds are irrelevant, while the readability and safety advantages are decisive. Error codes would be preferred only in a hot loop where most inputs are expected to be invalid.

Q2 – User-Defined Exceptions and Propagation in an Online Examination Portal (7 marks, K4)

Shared group work – hierarchy, checked/unchecked decision, three layers, chaining

The portal has three layers in package portal: **ExamController** (talks to the student), **SubmissionService** (business rules) and **Storage** (disk). Rules checked, in order: the arguments are valid → the student is registered → the exam window is open → the file is within 200 KB → the storage accepts it (total quota 500 KB).

```
package portal;
import java.time.LocalDateTime;
import java.util.Map;
import java.util.Set;
/** SERVICE layer: business rules. Declares (throws) what it cannot handle and lets it propagate upwards. */
public class SubmissionService {
    static final long SIZE_LIMIT_KB = 200;
    private final Map<String, Set<String>> registrations;     // course -> registered students
    private final Map<String, LocalDateTime> windowCloses;    // course -> deadline
    private final Storage storage;
    private final boolean lossy;

    public SubmissionService(Map<String, Set<String>> registrations, Map<String, LocalDateTime> windowCloses, Storage storage, boolean lossy) {
        this.registrations = registrations; this.windowCloses = windowCloses; this.storage = storage; this.lossy = lossy;
    }
    public void submit(String student, String course, String fileName, long sizeKb, LocalDateTime now) throws ExamException {
        if (student == null || student.isBlank() || fileName == null || fileName.isBlank() || sizeKb < 0)
            throw new InvalidSubmissionException("student, file name and a non-negative size are required");   // unchecked
        if (!registrations.getOrDefault(course, Set.of()).contains(student))
            throw new NotRegisteredException(student, course);
        LocalDateTime closes = windowCloses.get(course);
        if (closes == null || now.isAfter(closes))
            throw new ExamWindowClosedException(closes, now);
        if (sizeKb > SIZE_LIMIT_KB)
            throw new FileTooLargeException(fileName, sizeKb, SIZE_LIMIT_KB);
```

```
        if (lossy) storage.saveLossy(fileName, sizeKb); else storage.save(fileName, sizeKb);          //
SubmissionStorageException propagates
    }
}
```

```
package portal;
import java.io.IOException;
/** STORAGE layer: the only layer that knows about disks. Total disk quota 500 KB. */
public class Storage {
    private long usedKb = 0;
    private final long quotaKb = 500;
    public void save(String fileName, long sizeKb) throws SubmissionStorageException {
        try {
            if (usedKb + sizeKb > quotaKb) throw new IOException("Disk quota exceeded");    // the low-level
failure
            usedKb += sizeKb;
        } catch (IOException e) {
            throw new SubmissionStorageException("Could not store " + fileName, e);       // wrap, keep the cause
        }
    }
    /** Same failure, but the cause is thrown away (used to show what chaining protects). */
    public void saveLossy(String fileName, long sizeKb) throws SubmissionStorageException {
        try {
            if (usedKb + sizeKb > quotaKb) throw new IOException("Disk quota exceeded");
            usedKb += sizeKb;
        } catch (IOException e) {
            throw new SubmissionStorageException("Could not store " + fileName);          // cause LOST
        }
    }
}
```

Student A – Exception hierarchy and propagation design

A1. Class diagram of the exception hierarchy

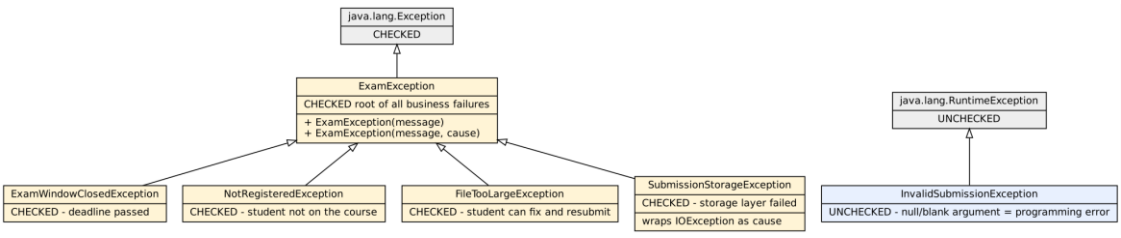


Figure Q2.1 – Exception hierarchy of the portal

A2. Why each exception is checked or unchecked

Exception	Kind	Reason
ExamException (root)	Checked	Gives the controller one type to catch for "a business rule or the storage failed" and forces every layer to decide what to do.
ExamWindowClosedException	Checked	An expected business event, not a bug. The student cannot fix it, but the caller must produce a proper message – the compiler should force that.
NotRegisteredException	Checked	Expected rejection that depends on data, not on a programming mistake.
FileTooLargeException	Checked	Recoverable: the student can compress the file and resubmit, so

		callers must handle it.
SubmissionStorageException	Checked	An environment failure (disk) that the controller can translate to "try again"; it **wraps** the IOException so the cause is kept.
InvalidSubmissionException	**Unchecked** (RuntimeException)	A blank student/file name is a **programming error** of the caller. Nobody can recover from it at run time, so it should not clutter every signature; it is fixed by correcting the code.

A3. Propagation path of one exception (storage → controller)

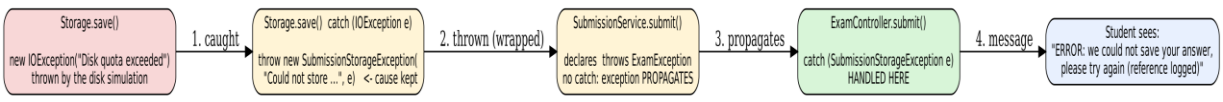


Figure Q2.2 – Path of a disk-quota failure through the three layers

Where it is finally handled and what the student sees: the IOException is caught in Storage.save, wrapped into a SubmissionStorageException (cause kept), and propagates through SubmissionService.submit (which only *declares* throws ExamException) to **ExamController.submit**, which has catch (SubmissionStorageException e). The controller is the correct place because it is the layer that talks to the

student and can choose the wording. The student sees: *"ERROR: we could not save your answer, please try again (reference logged)"* – no stack trace, no file-system detail. The original Disk quota exceeded stays available through getCause() for the log.

Student B – Failure scenarios and chaining evidence

B1. One scenario per exception – test table (observed behaviour)

#	Scenario	Exception that reached the controller	Message shown to the student
1	valid submission	none	OK: ans1.pdf submitted
2	student not registered	NotRegisteredException	REJECTED: you are not registered for CSA0928
3	window closed (12:01)	ExamWindowClosedException	REJECTED: the exam window has closed
4	exactly at closing time (boundary)	none	OK: ans3b.pdf submitted
5	file too large (201 KB)	FileTooLargeException	REJECTED: your file is larger than 200 KB - please compress it
6	file exactly at the limit (200 KB)	none	OK: ans4b.pdf submitted

7	disk quota exceeded (storage fails)	SubmissionStorageException	ERROR: we could not save your answer, please try again (reference logged)
8	blank file name (programming error)	InvalidSubmissionException	ERROR: invalid request

All 8 scenarios behaved as expected (Q2/output/PortalMain.txt). Scenarios 4 and 6 are **boundary** tests: submitting exactly at the closing time and a file of exactly 200 KB are accepted (isAfter and > are strict).

B2. Exception chaining: getCause() and the full chained stack trace

```
$ java -cp out portal.ChainDemo
--- WITH chaining (cause kept) ---
getMessage() = Could not store third.pdf
getCause()   = java.io.IOException: Disk quota exceeded
  level 0: portal.SubmissionStorageException : Could not store third.pdf
  level 1: java.io.IOException : Disk quota exceeded
full stack trace:
portal.SubmissionStorageException: Could not store third.pdf
    at portal.Storage.save(Storage.java:12)
    at portal.SubmissionService.submit(SubmissionService.java:26)
    at portal.ChainDemo.tryTwice(ChainDemo.java:20)
    at portal.ChainDemo.main(ChainDemo.java:31)
Caused by: java.io.IOException: Disk quota exceeded
    at portal.Storage.save(Storage.java:9)
    ... 3 more

--- WITHOUT the cause (re-thrown as a new exception) ---
getMessage() = Could not store third.pdf
getCause()   = null
  level 0: portal.SubmissionStorageException : Could not store third.pdf
full stack trace:
portal.SubmissionStorageException: Could not store third.pdf
```

Reading the chained trace: the first block is the SubmissionStorageException created in Storage.save and propagated up through SubmissionService.submit and ChainDemo; the line **Caused by: java.io.IOException: Disk quota exceeded** introduces the original exception with *its own* stack trace, which points to the exact line that detected the problem. ... N more means the remaining frames are identical to the enclosing exception's frames. getCause() returns that IOException; walking getCause() in a loop printed the two levels.

B3. What is lost if the exception is re-thrown without its cause

```
    at portal.Storage.saveLossy(Storage.java:21)
    at portal.SubmissionService.submit(SubmissionService.java:26)
    at portal.ChainDemo.tryTwice(ChainDemo.java:20)
    at portal.ChainDemo.main(ChainDemo.java:33)
```

Without the cause, getCause() is null, the trace has **no Caused by: section**, and the one fact that explains *why* storing failed – Disk quota exceeded – is gone; only "Could not store third.pdf" remains. The stack trace also no longer shows the line where the real problem was detected. In production this turns a 2-minute diagnosis into guesswork.

Joint conclusion – why a meaningful SubmissionStorageException instead of letting IOException travel to the UI?

(1) **Abstraction:** the controller and UI should not know that storage uses files; if storage moves to a database or cloud, IOException would become SQLException and every upper layer would have to change, whereas SubmissionStorageException stays. (2) **Meaning:** the exception name states the *business* failure ("the answer could not be stored") instead of a technical one. (3) **Safety:** a raw IOException message (paths, quotas) must not reach students. (4) **No information is lost** because the original is kept as the cause for the log. (5) The controller can handle the whole class of storage problems with one catch and one friendly message.

Complete the Code – StorageDemo

TODO 1 was replaced by one statement; the rest of the program is unchanged:

```
} catch (IOException e) {
    // TODO 1: Wrap the IOException in a SubmissionStorageException, keeping IOException as its cause
    throw new SubmissionStorageException("Could not store " + fileName, e);
}
```

Expected output	Actual output

Saved answer01.pdf Error: Could not store answer02.pdf Cause: Disk quota exceeded	Saved answer01.pdf Error: Could not store answer02.pdf Cause: Disk quota exceeded

What would the Cause: line print if the IOException were not passed to the constructor? getCause() would return null, so e.getCause().getMessage() is a call on null and throws a **NullPointerException** *inside the catch block* in main. There is no handler for it, so the program ends with a stack trace after printing Error: Could not store answer02.pdf – the Cause: line is never printed. Executed (StorageDemoNoCause):

```
$ java -cp out_nocause StorageDemoNoCause 2>&1; echo "exit code = $?"
Saved answer01.pdf
Error: Could not store answer02.pdf
Exception in thread "main" java.lang.NullPointerException: Cannot invoke "java.lang.Throwable.getMessage()" because the return value of "ExamException.getCause()" is null
    at StorageDemoNoCause.main(StorageDemoNoCause.java:33)
exit code = 1
```

Alternative Concept and Complexity Analysis – unchecked custom exceptions

If ExamException extended RuntimeException the classes would be the same except for the extends clause, but: (1) **method signatures** lose their throws clauses (the checked version has 3 throws declarations in Storage/SubmissionService; the unchecked copy in Q2/src/portal_unchecked has 0 – signature_comparison.txt); (2) **calling code** is no longer forced to handle or declare anything – the careless call in Q2/negative/ForgotToHandle.java is rejected by the compiler for the checked version (unreported exception ExamException; must be caught or declared to be thrown) but would compile silently for the unchecked version, so a forgotten catch surfaces only at run time as a crash; (3) the controller's catch blocks still work, but nothing guarantees they exist. Benefits of unchecked: shorter signatures, no changes to interface methods when a new failure is added. The same eight scenarios pass in both variants.

Time and space cost of capturing a stack trace (StackTraceCostBenchmark): creating an exception walks the stack, so time is Θ(d) in the depth d; chaining creates a *second* exception (the IOException) with its own trace, so the cost and memory roughly double.

Stack depth	**single exception (ns)**	**wrapper + cause (ns)**	**ratio**
5	450	988	2.20×

25	830	1706	2.06×
100	2842	5044	1.77×
400	9875	18377	1.86×

Stack depth	**memory: single (bytes)**	**memory: chained (bytes)**	**ratio**
5	719	1460	2.03×
25	732	1460	1.99×
100	2796	5588	2.00×

Both are **O(d)** in time and space (about 720 → 2,800 bytes for one exception from depth 5 to 100, about 1,460 → 5,590 for the chained pair). Chaining therefore costs a constant factor of about **2×** in time and memory, not a change of order. At an exam portal's failure rate this is negligible, and it buys the cause.

Q3 – Resource Safety, Debugging and Robust Code (7 marks, K4)

Shared group work – the generator in two versions (plus two bad ones for comparison)

The report generator reads the store files (input/stores: five files, each item,quantity,price per line) and writes one summary file (store1.csv,revenue … TOTAL,revenue). Four implementations of the **same** job are in ReportGenerator.java:

Version	How resources are closed
leaky() (the bug being investigated)	close() is the last statement of the try block – skipped when an exception occurs
manual() – required version 1	try / catch / finally with a closeQuietly() call in each finally
twr() – required version 2	try-with-resources for the writer and, inside the loop, for each reader
noClose() (alternative)	never closes anything

```
                throw new ReportException(f.getName(), e);
            }
        }
        total(out, total);
        out.close();                                // skipped on failure
    } catch (IOException e) { throw new ReportException(report.getName(), e); }
  }

  // ---------------------------------------------------------------- V1: try / catch / finally
  static void closeQuietly(Closeable c, String what) {
    if (c == null) return;
    try { c.close(); } catch (IOException e) { LOG.warning("close() failed for " + what + ": " +
e.getMessage()); }
  }
```

```
static void manual(List<File> stores, File report) throws ReportException {
    BufferedWriter out = null; double total = 0;
    try {
        out = new Tracked.Writer(report);
        for (File f : stores) {
            BufferedReader r = null;
            try {
                r = new Tracked.Reader(f);
                total += processStore(r, f.getName(), out);
            } catch (IOException e) {
                LOG.log(Level.SEVERE, "FAILED while reading " + f.getName() + ": " + e.getMessage());
                throw new ReportException(f.getName(), e);
            } finally {
                closeQuietly(r, f.getName());              // always runs
            }
        }
        total(out, total);
    } catch (IOException e) {
        throw new ReportException(report.getName(), e);
    } finally {
        closeQuietly(out, report.getName());               // flushes what was written so far
    }
}

// ---------------------------------------------------------------- V2: try-with-resources
static void twr(List<File> stores, File report) throws ReportException {
    double total = 0;
    try (BufferedWriter out = new Tracked.Writer(report)) {
        for (File f : stores) {
            try (BufferedReader r = new Tracked.Reader(f)) {
                total += processStore(r, f.getName(), out);
            } catch (IOException e) {
                LOG.log(Level.SEVERE, "FAILED while reading " + f.getName() + ": " + e.getMessage());
                throw new ReportException(f.getName(), e);
            }
        }
        total(out, total);
    } catch (IOException e) {
        throw new ReportException(report.getName(), e);
    }
```

```
    }
```

Both required versions produce the same report. ReportExperiment section 1 ran all versions on the good input; the SHA-256 hash of the summary file is identical and the program printed manual == try-with-resources report: true:

```
$ java -cp out ReportExperiment 2>&1
=== 1. SAME GOOD INPUT (input/stores: 5 files) - do all versions produce the same report? ===
version                  report hash  lines      tracked open   OS fds open
leaky (close in try)       22c5eda2     6          0              0
manual try/catch/finally   22c5eda2     6          0              0
try-with-resources         22c5eda2     6          0              0
never closed               22c5eda2     6          6              6
manual == try-with-resources report: true   (leaky and never-closed also equal on SUCCESS: true)
-> on a successful run the leaky version only looks fine because it closes at the end; the never-closed version leaks even on success.
```

Student A – Debugging and experimental analysis

A1. Locating the leak with breakpoints and step-through debugging

LeakDebugTarget runs the **leaky** version on the failing input. A scripted jdb session (Q3/debug_session.sh, compiled with javac -g) set two breakpoints: line 53, the r.close() call, and line 34, the throw in processStore. The transcript (full text in Q3/output/jdb_session.txt) shows the evidence:

```
Breakpoint hit: "thread=main", ReportGenerator.leaky(), line=53 bci=68
main[1]  f.getName() = "store1.csv"
main[1]  Tracked.OPEN.size() = 2
Breakpoint hit: "thread=main", ReportGenerator.leaky(), line=53 bci=68
main[1]  f.getName() = "store2.csv"
main[1]  Tracked.OPEN.size() = 2
Breakpoint hit: "thread=main", ReportGenerator.processStore(), line=34 bci=68
34                  throw new IOException("bad data in " + name + " line " + lineNo + ": \"" + line + "\"", e);
  [2] ReportGenerator.leaky (ReportGenerator.java:52)
  [3] LeakDebugTarget.main (LeakDebugTarget.java:6)
r = instance of Tracked$Reader(id=927)
name = "store3.csv"
lineNo = 3
line = "notebook,four,5.00"
```

```
main[1]  Tracked.OPEN.size() = 2
```

- The breakpoint on the close() line is hit for **store1.csv** and for **store2.csv** (Tracked.OPEN.size() is 2 at that moment – the summary writer and the current reader).
- For **store3.csv** execution stops at the throw in processStore (line 34) – where shows processStore ← leaky, locals shows lineNo = 3, and Tracked.OPEN.size() = 2.
- After cont, the close() breakpoint is **never hit again**: the exception left the try block, so r.close() and out.close() were skipped. That is the point where the manual version leaks.

A2. An input that makes the manual version fail part-way

input/stores_bad/ = the five good files with store3.csv replaced by four lines whose third line is notebook,four,5.00 (the quantity is not a number). Files 1 and 2 are processed normally, store3 fails after two good lines, stores 4 and 5 are never reached.

A3. Files left open in each version for the same failing input

```
version                 report hash  lines     tracked open   OS fds open    exception
[LOG] SEVERE: FAILED while reading store3.csv: bad data in store3.csv line 3: "notebook,four,5.00"
leaky (close in try)    e3b0c442     0         2              2              failed at store3.csv  leaked=[summary-writer, reader]
[LOG] SEVERE: FAILED while reading store3.csv: bad data in store3.csv line 3: "notebook,four,5.00"
manual try/catch/finally  c1387d2f   2         0              0              failed at store3.csv  leaked=[]
[LOG] SEVERE: FAILED while reading store3.csv: bad data in store3.csv line 3: "notebook,four,5.00"
try-with-resources      c1387d2f     2         0              0              failed at store3.csv  leaked=[]
[LOG] SEVERE: FAILED while reading store3.csv: bad data in store3.csv line 3: "notebook,four,5.00"
```

Version	**Handles still open after the failure**	**What the report on disk contains**
leaky (close in try)	**2** (the store3 reader and the summary writer) – confirmed also by the operating system (/proc/self/fd: 2)	**0 bytes** – the buffered writer was never flushed or closed, so the work of stores 1 and 2 is lost ("the report is

		sometimes incomplete")
manual try/catch/finally	**0**	Stores 1 and 2 (flushed by the finally)
try-with-resources	**0**	Stores 1 and 2
never closed	**3** (stores 1, 2, 3) + writer = 4 held open, never released	empty (not flushed on failure)

A4. Logging that records which file caused the failure

java.util.logging is used; every failure is logged at SEVERE with the file name, and the message of the IOException carries the line number. From the experiment output:

> [LOG] SEVERE: FAILED while reading store3.csv: bad data in store3.csv line 3: "notebook,four,5.00"
> [LOG] SEVERE: FAILED while reading store3.csv: bad data in store3.csv line 3: "notebook,four,5.00"
> [LOG] SEVERE: FAILED while reading store3.csv: bad data in store3.csv line 3: "notebook,four,5.00"
> [LOG] SEVERE: FAILED while reading store3.csv: bad data in store3.csv line 3: "notebook,four,5.00"

The ReportException carries the file name (e.file = store3.csv) so the caller can also report it.

Student B – Design and correctness

B1. Every resource opened and where it must be closed

Resource	**Opened**	**Must be closed**	**If it is not closed**
Summary BufferedWriter (+ the FileWriter it wraps)	once, before the loop	after the last store, **or on failure** – closing also	file handle leaked **and buffered lines lost** → incomplete report

		flushes the buffer	
Store BufferedReader (+ FileReader)	once per store file, inside the loop	as soon as that file is processed or fails	one handle per file; after ~1,000 files the process hits the OS limit → "Too many open files"

B2. Why the manual version can leak when an exception occurs

A close() written after the statements that can throw is simply **skipped** when control leaves the block through an exception; only code in a finally block runs on every path. The leaky version did exactly that (breakpoint evidence above). Even a finally version needs care: close() itself throws IOException, so a bare r.close(); out.close(); in one finally skips the second close if the first throws; and a finally that throws replaces the original exception (demonstrated next).

B3. How try-with-resources guarantees closing, and the order

The compiler turns try (R a = ...; R b = ...) { body } into nested try/finally code: every resource that was **successfully opened** is closed when the block ends – normally, by return, or by an exception – and **in the reverse order of opening**, so a resource that depends on an earlier one is closed first. If opening b fails, a is still closed. TwrDemo output:

```
=== 1. Closing order: reverse of opening ===
  open  A (opened first)
  open  B (opened second)
  body runs
  close B (opened second)
```

B4. Suppressed exceptions

When the body throws an exception **and** a close() also throws, only one exception can propagate. try-with-resources keeps the **body's** exception as the primary one and attaches the close() failures to it as **suppressed** exceptions (getSuppressed()). It occurs whenever both the body and a close() fail. With manual finally the exception from close() **replaces** the original – the original failure is lost.

```
=== 2. Body fails AND close() fails -> the close failure is SUPPRESSED, the body failure is reported ===
  open  A
  open  B
  close B
  close A
  caught   : body failed (the PRIMARY exception)
  suppressed (2): [java.lang.IllegalStateException: close() of B failed, java.lang.IllegalStateException: close() of A failed]


=== 3. The same situation with manual finally -> the ORIGINAL failure is lost ===
  open  A
  open  B
  close B
  close A
  caught   : close() of A failed   <- the body failure and B's close failure are GONE
  suppressed (0): nothing recorded
```

Joint engineering decision – when is try-with-resources not sufficient?

- **The number of resources is only known at run time** – a variable number of store files cannot all be declared in one try header. (Solved here by opening one reader at a time; if all had to be open together, a list plus a loop in finally, or a helper that closes a list and collects suppressed exceptions, is needed.)
- **The resource is not AutoCloseable** (legacy API with release()/disconnect()) – wrap it in an adapter.
- **close() failure must be handled, not just suppressed** – a failed close()/flush of the summary file means the report is silently incomplete; the exception thrown by close() of the *writer* on the success path must be checked and reported.
- **Atomicity:** closing a half-written report does not make it correct. A robust generator writes to a temporary file and renames it only on success, otherwise a reader sees a partial summary that looks valid.
- **Policy decisions** – continue with the other stores or abort? retry? roll back or compensate (locks, transactions, temp files)? These are not resource management.

- **Closing is not the same as flushing durably** (fsync), and a resource opened outside the try header (before it) is not covered.

Alternative Concept and Complexity Analysis – finally with explicit close(), or no closing at all

What the solutions are: manual() (explicit close() in finally) and noClose() are implemented and compared with try-with-resources on the same inputs. For f store files with m lines each:

	leaky	**manual finally**	**try-with-resources**	**never closed**
Time to process	Θ(f·m)	Θ(f·m)	Θ(f·m)	Θ(f·m)
Peak handles in a success run	2 (one reader + writer)	2	2	**f + 1**
Handles held open after a failure	2 (constant: the failing reader and the writer)	0	0	**up to f + 1** (all readers opened so far + writer)
Handles leaked per failure over a long-running service	grows by 2 each time	0	0	grows by f+1 each run

Measured (ReportScaleBenchmark; f files × 200 lines; time in ms, best of 5, and handles still open after a failure in the middle file):

f	leaky (ms)	manual (ms)	try-with-resources (ms)	never closed (ms)
10	7.4	4.5	4.4	2.1
100	18.4	12.4	8.6	5.7
1000	69.8	64.4	27.0	36.8

f	leaked: leaky	leaked: manual	leaked: try-with-resources	leaked: never closed
10	2	0	0	7
100	2	0	0	52
1000	2	0	0	502

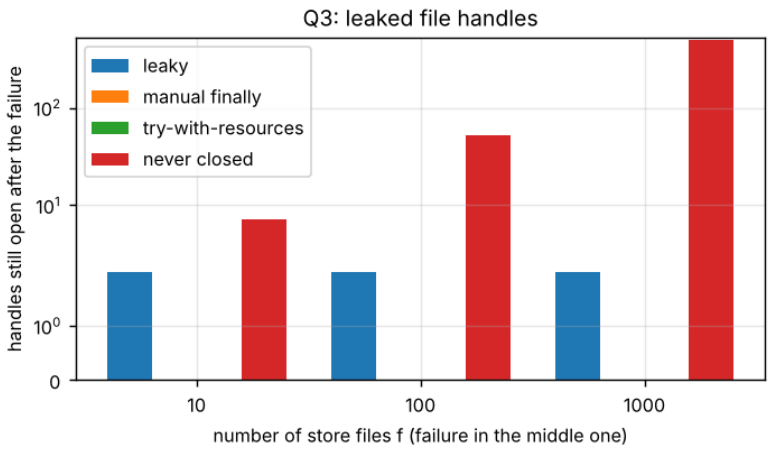


Figure Q3.1 – Handles still open after a failure (symlog axis)

Analysis. Time is Θ(f·m) for every version (all read every line once); the differences (e.g. 27 ms vs 65 ms at f = 1000) are noise and JIT effects, not complexity. The decisive difference is **space**: with explicit close in finally or with try-with-resources at most **2** handles are open at any time (O(1)) and **0** remain after a failure, whereas the never-closing version holds **f + 1** at the end of a success run and f/2 + 2 after a mid-run failure – O(f). That is the "Too many open files" bug, which I reproduced by running the never-closing version under ulimit -n 128 with 400 files:

```
$ bash -c "ulimit -n 128; java -cp out ReportScaleBenchmark ulimit" 2>&1 | head -3
FAILED after 122 handles were open: /tmp/stores_400_727185945210631951/store00121.csv (Too many open files)
```

The failure came after 122 open handles (the JVM itself needs a few of the 128 descriptors). The leaky version leaks only 2 per failure, so it fails only after many failed runs in a long-lived process, which is why the bug appears "occasionally". Try-with-resources is retained: same time, O(1) space, no leak, and the suppressed-exception rule keeps the original failure visible.

Q4 – Thread Creation and the Thread Life Cycle (7 marks, K4)

Shared group work – two ways to create the stations

Four scanner stations; each scans a batch of 20 scripts (1–4 ms per script, simulated with Thread.sleep) and the main thread prints the finishing order and the total (expected 4 × 20 = 80). **Way 1** StationThread extends Thread, **way 2** StationRunnable implements Runnable run by new Thread(task, name); both use start(), sleep(), join() and isAlive().

```
/** Way 1: EXTEND Thread. */
public class StationThread extends Thread {
   private final int batch;
   volatile int scanned;
   StationThread(String name, int batch) { super(name); this.batch = batch; }
   @Override public void run() {
      scanned = Scanner1.scan(getName(), batch);
      StationsMain.FINISHED.add(getName());
   }
}


/** Way 2: IMPLEMENT Runnable (the task is separate from the thread that runs it). */
public class StationRunnable implements Runnable {
   private final String name; private final int batch;
   volatile int scanned;
   StationRunnable(String name, int batch) { this.name = name; this.batch = batch; }
   @Override public void run() {
      scanned = Scanner1.scan(name, batch);
      StationsMain.FINISHED.add(name);
   }
}
```

```
$ java -cp out StationsMain thread; java -cp out StationsMain runnable
alive right after start(): true true true true
alive after join()       : false false false false
finish order: S2 S1 S3 S4   total scripts scanned = 80
finish order: S4 S2 S1 S3   total scripts scanned = 80
```

Student A – Thread design

A1. Extending Thread vs implementing Runnable

	extends Thread	**implements Runnable**
What it is	The task **is** a thread	The task is separate from the thread that runs it
Inheritance	Uses up the single superclass slot	Class can still extend another class
Reuse	One object = one thread; cannot be handed to an executor as a plain task	The same task can run in a Thread, an ExecutorService, or be run directly; one task object can be shared
Design	Mixes *what* to do with *how* it runs	Separates job from execution mechanism (cleaner, testable)
Preferred when	Rarely – only if the thread behaviour itself (e.g. an overridden interrupt()) is being specialised	**Normally**: it is the preferred way and the basis for Callable/executors

A2. Life-cycle state diagram – states each station passes through

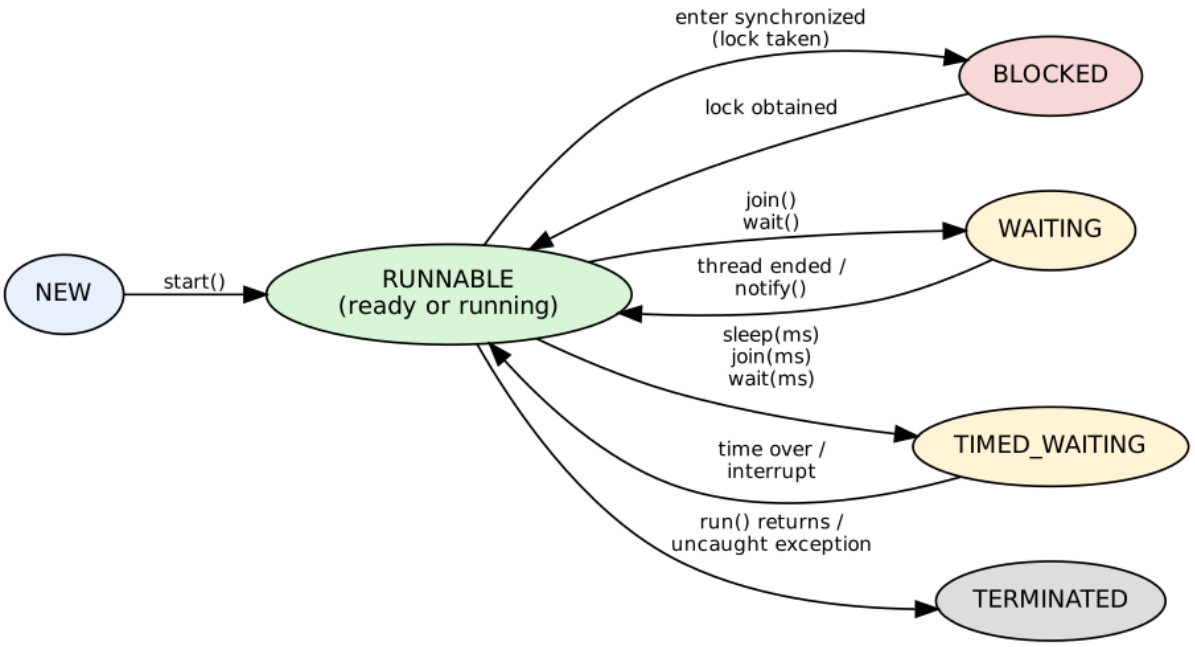


Figure Q4.1 – Thread life cycle (Thread.State). A station passes NEW → RUNNABLE → TIMED_WAITING (every sleep) ↔ RUNNABLE → TERMINATED; main passes through WAITING while in join().

LifeCycleDemo observed one thread in every state with getState():

```
after new Thread(...)          : NEW   isAlive=false
```

```
50 ms after start(), computing : RUNNABLE   isAlive=true
while inside Thread.sleep(300) : TIMED_WAITING
after the run() method ended   : TERMINATED   isAlive=false
waiting to enter synchronized  : BLOCKED
inside wait()                  : WAITING
inside join() of another thread: WAITING
```

A3. Why run() does not create a new thread – evidence

run() is an ordinary method: calling t.run() executes the body **in the calling thread**; only start() asks the JVM to create a new thread, which then calls run().

```
$ java -cp out RunVsStart
main thread id = 1
calling t.run():
  body executed by thread: main  (id 1)
  t1 state after run(): NEW   (the thread was never started)
calling t.start():
  body executed by thread: station-2  (id 14)
starting the same thread twice:
  java.lang.IllegalThreadStateException
four 100 ms jobs: via run() = 401 ms (sequential, one after another), via start() = 100 ms (overlapping)
```

Evidence: with run() the body printed main as the thread name and the thread stayed in state NEW; with start() the name was station-2; starting the same thread twice throws IllegalThreadStateException; and four 100 ms jobs took about 400 ms through run() (one after another) versus about 100 ms through start() (overlapping).

A4. How join() ensures the final total is printed after all stations finish

t.join() makes the calling thread (main) wait – state WAITING – until t has terminated. Calling join() on all four stations before printing means main cannot reach the print statement until every station has finished and stored its count.

```
$ java -cp out JoinDemo
--- without join() ---
total printed immediately by main : 0   (WRONG - stations still scanning)
total after the stations finished  : 80
--- with join() ---
total printed after join()         : 80   (correct: 4 x 20 = 80)
```

Without join() main printed a total of 0 immediately (wrong); with join() it printed 80. join() also gives a **happens-before** guarantee, so main sees the stations' results (scanned) correctly.

Student B – Behavioural experiments

B1. Finishing order over 12 runs (separate JVMs, both ways of creating threads)

Run	Finish order – extends Thread	Finish order – implements Runnable
1	S3 S1 S4 S2	S3 S2 S1 S4
2	S4 S1 S3 S2	S2 S4 S3 S1
3	S4 S1 S3 S2	S4 S1 S2 S3
4	S4 S2 S1 S3	S1 S3 S4 S2
5	S1 S3 S4 S2	S4 S1 S2 S3
6	S4 S2 S1 S3	S4 S2 S1 S3
7	S4 S3 S2 S1	S2 S3 S1 S4
8	S4 S2 S3 S1	S4 S3 S1 S2
9	S3 S4 S2 S1	S4 S3 S2 S1
10	S4 S2 S3 S1	S4 S2 S1 S3
11	S2 S4 S1 S3	S4 S3 S1 S2
12	S1 S2 S4 S3	S2 S4 S1 S3

The total was **80 in all 24 runs** (Q4/output/twelve_runs.txt), but the order differs from run to run and there is no stable pattern: S4 finished first in 7 of the 12 Thread runs, S1 and S3 in 2 each and S2 in 1; in the Runnable runs the first finisher was S3, S2, S4, S1, S4, S4, S2, S4, S4, S4, S4, S2. (S1 has no systematic advantage even though it is started first.)

B2. Thread priorities

PriorityExperiment: four CPU-bound stations, 12 repetitions per setting.

```
all priority 5:
times each station finished FIRST: S1=9 S2=0 S3=2 S4=1
times each station finished LAST : S1=0 S2=3 S3=4 S4=5

S1 = 10 (MAX), S2 = S3 = 5, S4 = 1 (MIN):
times each station finished FIRST: S1=6 S2=3 S3=1 S4=2
times each station finished LAST : S1=1 S2=2 S3=1 S4=8
```

Did the order change? Not in the way priorities promise. With equal priorities S1 finished first in 9 of 12 runs (it simply started first and got a head start). With S1 at MAX it finished first in only **6** of 12 – *fewer* – and S4 at MIN finished last 8 times against 5 (a small shift that is within the run-to-run noise of 12 samples). On Linux HotSpot (ThreadPriorityPolicy = 0 in this JVM) Java priorities are **only hints that the operating system scheduler is free to ignore**, and the machine has a single core, so threads are time-sliced regardless of priority. A program must never use priorities for correctness.

B3. Daemon thread when main() ends

```
$ java -cp out DaemonDemo daemon; echo '--- same program, NOT daemon ---'; java -cp out DaemonDemo
S4 isDaemon = true
  S4 scanned script 1
  S4 scanned script 2
main() is ending now
--- same program, NOT daemon ---
S4 isDaemon = false
  S4 scanned script 1
  S4 scanned script 2
main() is ending now
  S4 scanned script 3
  S4 scanned script 4
  S4 scanned script 5
  S4 scanned script 6
  S4 scanned script 7
  S4 scanned script 8
  S4 scanned script 9
  S4 scanned script 10
  S4 scanned script 11
  S4 scanned script 12
  S4 scanned script 13
  S4 scanned script 14
  S4 scanned script 15
  S4 scanned script 16
  S4 scanned script 17
  S4 scanned script 18
  S4 scanned script 19
  S4 scanned script 20
  S4 FINISHED its whole batch
```

S4 is a long task (20 scripts × 50 ms = 1 s); main joins only S1 (≈ 50 ms) and then ends. When S4 is a **daemon** (first run) the JVM exits as soon as the last non-daemon thread (main) ends: S4 printed only two scripts and

never printed "FINISHED" – its work was abandoned. When S4 is a normal (non-daemon) thread (second run) the JVM stays alive until S4 completes all 20 scripts and prints "S4 FINISHED". A daemon is therefore suitable for background housekeeping, never for work that must complete.

B4. Explanation with the behaviour of the thread scheduler

start() only makes a thread *runnable*; **which runnable thread gets the CPU, when, and for how long** is decided by the operating-system scheduler (time-slicing, interrupts, other processes). The stations sleep for random 1–4 ms periods, so each wake-up depends on timer resolution and on when the scheduler next runs that thread; small timing differences accumulate into different finishing orders. Priorities are hints, the scheduler is not obliged to run a thread just because it was started first, and the order cannot be predicted from the source code.

Joint conclusion – why different output orders, and why a correct program must not depend on the order

The same program produces different orders because the scheduler's decisions depend on things the program does not control (system load, timer granularity, number of cores, JIT warm-up, the JVM and OS version) – the 24 recorded runs above show it. A program whose correctness depends on one order will work on one machine on one day and fail elsewhere or rarely – the hardest kind of defect to reproduce. Correctness must therefore come from **explicit coordination** (join(), locks, Future.get(), blocking queues), exactly as join() guarantees the total here regardless of the finishing order.

Alternative Concept and Complexity Analysis – ExecutorService with a fixed pool

What the solution would be. The stations become tasks submitted to Executors.newFixedThreadPool(4); a Callable<Integer> returns the count and Future.get() replaces join():

```
ExecutorService pool = Executors.newFixedThreadPool(4);
List<Future<Integer>> fs = new ArrayList<>();
for (int s = 1; s <= 4; s++) { final int id = s; fs.add(pool.submit(() -> Scanner1.scan("S" + id, 20))); }
int total = 0; for (Future<Integer> f : fs) total += f.get();      // waits, like join(), AND returns the result
pool.shutdown(); pool.awaitTermination(5, TimeUnit.SECONDS);
```

Completion time. With n tasks of duration t on p threads, tasks run in ⌈n/p⌉ batches, so the time is about ⌈n/p⌉·t – Θ(n/p) while p ≤ the number of tasks that can really run at once. Measured for n = 400 scripts of 5 ms (ExecutorStations, Figure Q4.2):

p (pool threads)	model ⌈n/p⌉ × 5 ms	measured (ms)	efficiency

1	2000	2069	0.97
2	1000	1032	0.97
4	500	514	0.97
8	250	257	0.97
16	125	132	0.94
50	40	48	0.83

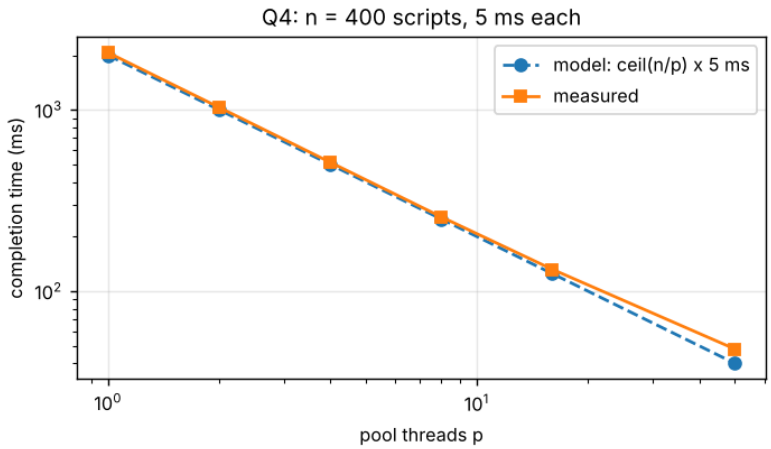


Figure Q4.2 – Pool completion time vs the n/p model

The measurements follow the model to within 3 % up to p = 8 (efficiency 0.97) and degrade at p = 50 (0.83) because 50 threads share one core and the scheduling overhead becomes visible. The four-thread version of the assignment is just n = 4 tasks on p = 4: one batch.

Memory per thread. Every platform thread reserves a stack of 1,024 KB by default (-XX:ThreadStackSize=1024). Measured with 500 parked threads: reserved virtual memory +471040 KB (**942 KB per thread**, i.e. the 1 MB stack reservation) but only +38180 KB **resident** (**76 KB per thread** – stack pages are committed lazily). Hand-made threads: one per task = O(n) threads × stack size (for n = 10,000 scripts: about 10 GB reserved). Fixed pool: O(p) threads, i.e. 4 × 1 MB, **independent of the number of tasks**. Creation cost: 2,000 tiny tasks took 85.0 ms with a new Thread each (42.5 µs per task) versus 1.2 ms on a pool of 4 (0.6 µs per task) – about 70× cheaper because the pool re-uses its threads. The pool is retained for any realistic number of scripts; hand-made threads are acceptable only for a handful of long-lived tasks.

Q5 – Race Conditions in a Cinema Seat-Booking System (6 marks, K4)

Complete the Code – SeatBooking

The assignment code has no book method at all, so TODO 1 is the whole method. **First version, WITHOUT synchronized** (SeatBookingUnsync.java, otherwise identical), then the correct one:

```
// TODO 1 - version 1 (no synchronization)          // TODO 1 - version 2 (correct)
void book(int seat) {                               synchronized void book(int seat) {
   if (!booked[seat]) {                               if (!booked[seat]) {      // check ...
      booked[seat] = true;                                booked[seat] = true;  // ... then act, as ONE indivisible step
      bookingsMade++;                                     bookingsMade++;
   }                                                  }
}                                                   }
```

Result of 10 runs of the unsynchronized version

Run	Unsynchronized (as given): Seats sold	Unsynchronized + pause between check and book: Seats sold
1	10	17
2	10	15
3	10	17
4	10	15
5	10	16
6	10	12
7	10	16
8	10	17
9	10	15
10	10	18

Honest observation: the program exactly as given printed Seats sold: 10 in **all 10 runs** on this machine – the race exists in the code but its window (a few nanoseconds between the if and the assignment) is almost never

hit when the 100 threads are started one after another on a single core. That is why such bugs reach production. To see the race the check and the update must be separated by an operation that lets another thread run (a database call or payment step in a real system); with Thread.yield() inserted between them (SeatBookingUnsyncSlow) the program sold **12–18 seats** for 10 seats in every run. With the synchronized method (TODO 1 version 2) the program printed the expected output:

```
$ java -cp out SeatBooking
Seats sold: 10
```

Why the unsynchronized version may sell more than 10 seats: book is a *check-then-act* sequence. Two threads can both execute the check while the seat is still free, and both then book it; each also increments bookingsMade, so the counter exceeds the number of seats (and because bookingsMade++ is itself a read-modify-write, increments can also be lost). Synchronization makes check and update one atomic unit with respect to other threads using the same lock.

Student A – Producing and explaining the race

A1. A test that reliably reproduces double booking

RaceReproducer: 8 threads wait at a CyclicBarrier and are released together to book the **same** seat; a trial with more than one "granted" is a double booking. 500 trials per batch, 5 batches per strategy. Thread.yield() between check and set widens the window (the natural window is too small on one core):

Strategy	**batch 1**	**batch 2**	**batch 3**	**batch 4**	**batch 5**	**total**	**max granted**
unsynchronized (natural race)	0	0	0	0	0	0 / 2500	1
unsynchronized (window widened)	499	499	500	500	500	2498 / 2500	8
synchronized method	0	0	0	0	0	0 / 2500	1
synchronized block	0	0	0	0	0	0 / 2500	1

Double bookings over repeated runs: 2,498 of 2,500 trials with the widened window (up to 8 customers got the same seat); **0 of 2,500** with the natural window on this machine (reported as observed – the natural race did not fire in 2,500 trials); **0 of 2,500** for both synchronized versions.

A2. The race condition step by step (two threads, seat 3)

Step	**Thread T1**	**Thread T2**	**booked[3]**	**bookingsMade**

1	reads booked[3] → false (seat free)		false	0
2	*(pre-empted here)*	reads booked[3] → **false** (still free!)	false	0
3		sets booked[3] = true	true	0
4		bookingsMade++ → 1	true	1
5	*(resumes with its old answer "free")* sets booked[3] = true		true	1
6	bookingsMade++ → 2		true	**2**

Both customers were told the seat is theirs, and the counter says 2 sales for 1 seat.

A3. The exact critical section

```
if (!booked[seat]) {        // read  booked[seat]
   booked[seat] = true;    // write booked[seat]       } these three statements form the critical section:
   bookingsMade++;         // read-modify-write        } shared mutable state, check-then-act
```

Everything else in a booking (computing the price, charging the customer, sending the SMS) touches no shared data and does **not** belong inside the critical section.

Student B – Comparing synchronization strategies

B1. Time to process 10,000 bookings (8 threads, 1,000 seats, each requested 10 times; best of 5)

Strategy	**no external work (ms)**	**with 100 µs payment step per booking (ms)**	**seats sold (must be 1000)**
synchronized method	0.4	1660.5	1000 / 1000
synchronized block	0.4	203.8	1000 / 1000
ReentrantLock	0.5	205.2	1000 / 1000
AtomicBoolean[] CAS	0.3	205.5	1000 / 1000
ConcurrentHashMap	0.6	205.0	1000 / 1000

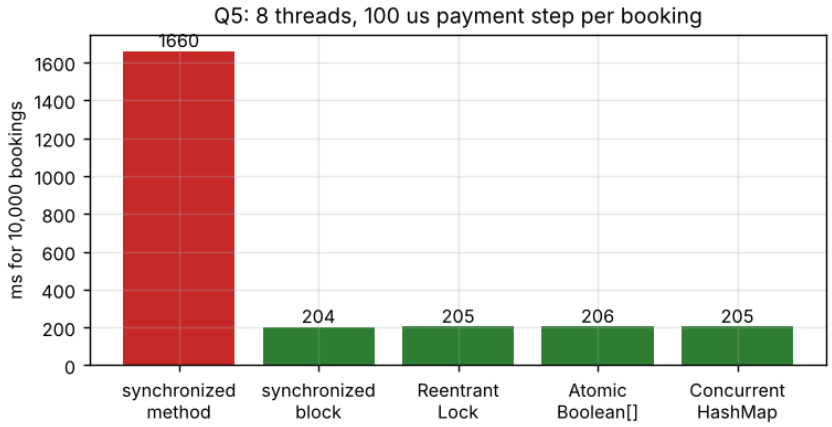


Figure Q5.1 – Locking the whole method vs only the critical section

B2. Locking the whole method vs only the critical section

When the booking does nothing but check-and-set (no external work) all strategies take well under a millisecond for 10,000 bookings (0.3–0.6 ms; the time is dominated by creating threads), so the lock scope does not matter. When each booking also performs a 100 µs step that needs no shared data (payment gateway, SMS), the **synchronized method holds the lock during that step**, so the 10,000 steps run one after another: **1,660 ms**. With a **synchronized block** around only the check-and-set the steps overlap (the threads wait concurrently): **204 ms – about 8× faster** with 8 threads, with the same correctness (1,000 seats sold, none twice). Rule: **hold a lock for as short a time as possible, and never while waiting for I/O.**

B3. Static synchronization and when it would be needed here

A synchronized **instance** method locks this; a static synchronized method locks the Class object, **one lock for the whole class**. It would be needed here for state shared by **all screens**, e.g. a static counter of tickets sold campus-wide. StaticSyncDemo shows the difference: 4 screens (4 objects, so 4 different this locks) each add 20,000 to one static counter (expected 80,000):

```
=== 4 screens, each booking 20,000 times into ONE static counter (expected 80000) ===
  instance-synchronized (4 different locks): 20000
  instance-synchronized (4 different locks): 20002
  instance-synchronized (4 different locks): 20000
  static synchronized   (one Class lock)   : 80000
  static synchronized   (one Class lock)   : 80000
  static synchronized   (one Class lock)   : 80000
(Thread.yield() between the read and the write of the counter widens the window so the lost updates are visible on one core)
```

The instance-synchronized version lost most updates (20,000 instead of 80,000: each screen's lock protects only itself) whereas static synchronized gave exactly 80,000 every time (Thread.yield() between the read and the write widens the window so it is visible on one core). The per-screen booked[] array needs **no** static lock – it belongs to one screen.

B4. The intrinsic lock each version uses

```
=== Which intrinsic lock does each form use? ===
  synchronized method      holds this lock         : true   holds class lock: false
  synchronized (this)      holds this lock         : true
  synchronized (private)   holds the private lock  : true   holds this lock: false
  static synchronized      holds the Class lock    : true
```

Version	Intrinsic lock (monitor)
synchronized void book()	this – the SeatBooking object of that screen
synchronized (lock) { ... } block	the private lock object (any object); this could be used too
static synchronized	the Class object (SeatBooking.class) – shared by all instances
ReentrantLock (alternative)	an explicit lock object, not the intrinsic monitor

Joint engineering decision – which strategy for a cinema with 20 screens?

One lock per screen: each screen is its own SeatBooking object, so a synchronized instance method or block locks only that screen; bookings on different screens never wait for each other (a single global or static lock would serialise all 20 screens for no reason). **Inside a screen, lock only the check-and-set** (synchronized block) and do the payment and notification outside the lock – measured 8× faster than locking the whole method when each booking has a 100 µs external step. AtomicBoolean.compareAndSet is a good alternative if the same seat is requested extremely often (measured 8–12 ns per booking against 14–23 ns), but it needs one object per seat (20 bytes against 1 byte per seat). Rules for deployment: never hold the lock while calling out of the process, keep the critical section to three statements, and use AtomicInteger/LongAdder for any campus-wide static counter.

Alternative Concept and Complexity Analysis – AtomicBoolean[], ReentrantLock, ConcurrentHashMap

```
// AtomicBoolean[] with compareAndSet          // ReentrantLock                    // ConcurrentHashMap.putIfAbsent
boolean book(int seat) {                       lock.lock();                     boolean book(int seat) {
  if (booked[seat].compareAndSet(false, true)) {  try { if (!booked[seat]) {...} }       return map.putIfAbsent(seat, TRUE) ==
null;
     sold.incrementAndGet(); return true; }     finally { lock.unlock(); }            }
  return false; }
```

Time per booking under contention (400,000 attempts, 8 threads; HOT = 10 seats so many threads compete for the same seats, SPREAD = 100,000 seats so they rarely meet), ns per booking, best of 5, and **extra space for s seats**:

Strategy	**HOT (ns)**	**SPREAD (ns)**	**Extra space for s seats**
synchronized method	15	14	s bytes (boolean[]) + O(1) lock
synchronized block	14	17	s bytes + O(1) lock
ReentrantLock	23	23	s bytes + one lock object (~48 bytes)
AtomicBoolean[] CAS	8	12	≈ 20 bytes per seat (AtomicBoolean object + reference) = 20·s
ConcurrentHashMap	10	66	≈ 58 bytes per sold seat (node + Integer key + table slot) = O(s)

Analysis. Every strategy performs a booking in O(1) time. Under contention the compare-and-set version is fastest on a hot seat (8 ns) because a loser fails immediately without blocking; the lock-based versions pay for lock hand-over (14–23 ns). ConcurrentHashMap is cheap for few keys (10 ns) but slows to 66 ns when the map holds 100,000 entries (hashing, node allocation, cache misses). Space: boolean[] guarded by one lock is the smallest (s bytes); AtomicBoolean[] costs ≈ 20 s bytes; ConcurrentHashMap ≈ 58 s bytes. On this single-core machine true parallel contention cannot occur, so absolute numbers should be re-measured on a multi-core computer; the **ordering of space costs and the lock-scope result are architecture-independent**.

Q6 – Deadlock and Inter-Thread Communication (7 marks, K4)

Complete the Code – TokenCounter (Part B)

Note: the printed code is missing the closing brace of the lambda of the student threads (it appears detached at the end of the table in the assignment); one } was added so that the program compiles. Nothing else outside the TODOs changed.

```
public class TokenCounter {
    private final int[] buffer = new int[5]; // the counter holds at most 5 tokens
    private int count = 0, in = 0, out = 0;

    public synchronized void put(int token) throws InterruptedException {
        // TODO 1: Wait while the counter is full, add the token at position in, then
        // wake waiting threads
        while (count == buffer.length) wait();      // loop: the condition is re-tested after every wake-up
        buffer[in] = token;
        in = (in + 1) % buffer.length;
        count++;
        notifyAll();
    }

    public synchronized int take() throws InterruptedException {
        // TODO 2: Wait while the counter is empty, remove the token at position out,
        // wake waiting threads and return the token
        while (count == 0) wait();
        int token = buffer[out];
        out = (out + 1) % buffer.length;
        count--;
        notifyAll();
// ... (26 more lines in the repository)
```

Expected output	**Actual output**
Sum of tokens collected: 210 (expected 210)	Sum of tokens collected: 210 (expected 210)

Change while to if in both methods and run several times

Run	Sum of tokens (expected 210)
1	175
2	210
3	210
4	185
5	175
6	175
7	200
8	200
9	200
10	60

With if, **8 of the 10 runs produced a wrong sum** (175, 185, 200, 60 …) while while always gave 210. **Explanation:** a thread that waited in if (count == 0) wait(); continues *without testing again* after it is woken. With notifyAll() both consumers wake up; the first takes the only token, the second then also proceeds although count is 0 again – it removes a token that is not there (it re-reads a stale slot and drives count to −1). Wrong sums, repeated and missing tokens, and a counter outside 0..5 follow. Spurious wake-ups (allowed by the Java specification) cause the same error even with a single consumer.

Student A – Producer–consumer design

A1. Why wait() must be in a loop and in a synchronized block

- **Synchronized:** wait() releases the monitor and later re-acquires it, and the condition (count == 0) is shared state that must be tested while holding that same lock, otherwise a wake-up could be lost between the test and the wait. A call without owning the monitor throws IllegalMonitorStateException.
- **Loop:** after being woken the thread must **re-test the condition**, because (1) another thread may have changed it first (the two-consumer case above), (2) notifyAll() wakes *every* waiter although only one can proceed, and (3) the JVM may wake a thread spuriously. while is therefore the only correct form; evidence: 8 wrong runs of 10 with if, none with while.

A2. notify() instead of notifyAll() with several consumers

CounterStress (10 trials each, 2-second watchdog; HUNG = threads still blocked):

Configuration	**Outcomes of 10 trials**
while, capacity 5, 1 producer, 2 consumers (the assignment)	OK=10
if, capacity 5, 1 producer, 2 consumers (the assignment)	OK=7, SIZE OUT OF RANGE=3
while, capacity 1, 2 producers, 4 consumers, 500 tokens each	OK=10
if, capacity 1, 2 producers, 4 consumers, 500 tokens each	SIZE OUT OF RANGE=10
notifyAll, 1 producer, 2 consumers, capacity 5	OK=10
notify, 1 producer, 2 consumers, capacity 5	OK=10
notifyAll, 2 producers, 4 consumers, capacity 1	OK=10
notify, 2 producers, 4 consumers, capacity 1	HUNG=10

In the assignment's design (1 producer, 2 consumers, capacity 5) notify() worked in all 10 trials: the waiting set contains threads of **one kind at a time** (the producer waits only when the buffer is full, consumers only when it is empty – never both), so waking any one waiter always helps. **With several producers and several consumers** (2 producers, 4 consumers, capacity 1) notify() **hung in all 10 trials**: a consumer's notify() can wake another *consumer*, which finds the buffer empty and waits again, while the producer that should have been woken stays asleep – the wake-up is consumed by the wrong thread and every thread ends up waiting. notifyAll() is correct in both cases because all waiters re-test their condition.

A3. Counter size over time – never above 5 or below 0

```
$ java -cp out CounterTrace
counter size after each of the 40 operations (put or take):
[1, 2, 3, 4, 5, 4, 3, 4, 5, 4, 5, 4, 5, 4, 5, 4, 5, 4, 3, 4, 5, 4, 5, 4, 5, 4, 3, 4, 5, 4, 5, 4, 5, 4, 3, 2, 1, 0]
minimum size seen = 0, maximum size seen = 5  (limits: 0 and 5)
sum of tokens = 210 (expected 210), final size = 0
```

The size was recorded after each of the 40 operations: it stays between 0 and 5 (minimum 0, maximum 5) and the sum is 210. In the stress test with if, the same monitor reported sizes outside the range (see the table), which is how the defect was detected.

A4. sleep() versus wait()

```
holder thread is inside synchronized(LOCK) for 400 ms; another thread tries to enter 100 ms later:
  holder uses Thread.sleep(400): the other thread got in after 300 ms   (sleep KEEPS the lock)
  holder uses LOCK.wait(400)   : the other thread got in after 0 ms     (wait RELEASES the lock)
```

	Thread.sleep(ms)	**Object.wait()**
Defined in	Thread (static)	Object (instance, on the monitor)
Lock	**keeps** every lock it holds (the other thread entered after 300 ms)	**releases** the monitor (the other thread entered after 0 ms)
Must be in synchronized?	No	Yes
Wakes up	when the time is over (or interrupt)	on notify()/notifyAll(), timeout (if given), or interrupt
Purpose	pause	wait for a **condition** to change

Student B – Deadlock analysis and prevention (Part A)

B1. A pair of transfers that reliably deadlocks

transferUnsafe(from, to) locks from, pauses 200 ms (so the bad interleaving is certain), then locks to. T1 = A → B and T2 = B → A:

```
Thread t1 = new Thread(() -> Wallet.transferUnsafe(a, b, 100, 200), "Transfer-1 (A -> B)");
Thread t2 = new Thread(() -> Wallet.transferUnsafe(b, a,  50, 200), "Transfer-2 (B -> A)");
```

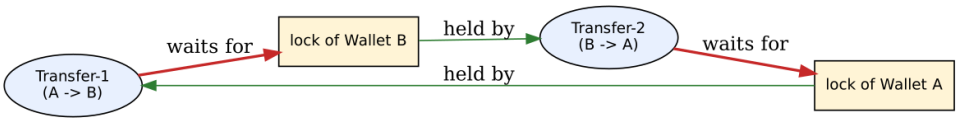


Figure Q6.1 – Wait-for graph: each transfer holds one wallet and waits for the other (a cycle)

```
$ java -cp out DeadlockDemo
DEADLOCK DETECTED among 2 threads:
  thread  : Transfer-1 (A -> B)   state = BLOCKED
    waiting for lock : Wallet@4dd8dc3
    which is held by : Transfer-2 (B -> A)
  thread  : Transfer-2 (B -> A)   state = BLOCKED
    waiting for lock : Wallet@6f496d9f
    which is held by : Transfer-1 (A -> B)
balances unchanged: A=1000 B=1000  (neither transfer completed; both threads wait forever)
t1 alive=true t2 alive=true -> exiting the JVM with System.exit(0)
```

ThreadMXBean.findDeadlockedThreads() reported both threads in state BLOCKED, each waiting for the lock the other owns; neither transfer completed (balances unchanged) and the program had to be ended with System.exit.

B2. The four conditions for deadlock present in the program

Condition	**Present in the program?**
Mutual exclusion – a resource is held by one thread at a time	Yes – a wallet's monitor can be owned by only one thread
Hold and wait – a thread holds a resource while waiting for another	Yes – each transfer holds the first wallet while waiting for the second
No pre-emption – a held resource cannot be taken away	Yes – a monitor cannot be taken from its owner
Circular wait – a cycle of threads each waiting for the next	Yes – T1 → B (held by T2) and T2 → A (held by T1)

B3. Prevention with a consistent lock-ordering rule, and why it works

```
static void transferOrdered(Wallet from, Wallet to, long amount) {
    Wallet first  = from.id < to.id ? from : to;      // ALWAYS lock the wallet with the smaller id first
    Wallet second = from.id < to.id ? to : from;
    synchronized (first) { synchronized (second) { from.balance -= amount; to.balance += amount; } }
```

```
}
```

Why it works: the rule breaks the fourth condition, **circular wait**. All threads acquire locks in one global order (increasing id), so a thread holding wallet *i* can only ever wait for a wallet with a **larger** id; a cycle would need some thread to wait for a smaller id, which the rule forbids. Hence the wait-for graph cannot contain a cycle. Both original transfers now both lock wallet 1 first: one of them waits briefly and then proceeds.

```
$ java -cp out DeadlockFixed
=== The two conflicting transfers with lock ordering (lower wallet id locked first) ===
both transfers completed: A=950 B=1050   (expected A=950, B=1050)



=== Stress: 8 threads x 200 random transfers between 6 wallets, 1 ms pause between the two locks ===
unsafe order (from, to)          HUNG (deadlock)  money before=60000 after=60000 (conserved)  time=5011 ms
ordered locks (lower id first)    finished        money before=60000 after=60000 (conserved)  time=641 ms
tryLock with timeout + back-off   finished        money before=60000 after=60000 (conserved)  time=1508 ms
```

Both transfers completed (A = 950, B = 1050). In the stress test (8 threads, random transfers between 6 wallets, 1 ms pause between the two locks) the unsafe order **hung** (deadlock found after the 5-second watchdog), while ordered locking finished and the total money was conserved (60,000 before and after).

Joint conclusion – what deadlock and producer–consumer have in common, and how each ensures progress

Both are **coordination problems between threads that wait for each other**: threads block until a condition that *another thread* must establish (a lock released, a slot freed, a token produced). Both can leave all threads waiting forever if the design is wrong (circular wait; a lost or wrongly targeted wake-up – the notify() hang above). **Progress** is ensured differently: the wallet design guarantees progress by **removing the cycle** (a global lock order, so some thread can always acquire its second lock); the token counter guarantees progress by the **condition protocol** – every state change (put, take) calls notifyAll(), every waiter re-tests its condition in a while loop, and a full counter can always be emptied by consumers while an empty one can be refilled by the cook.

Alternative Concept and Complexity Analysis – ArrayBlockingQueue and tryLock with a timeout

Part B with ArrayBlockingQueue<Integer>(5) (TokenQueueDemo) – no wait/notify code at all; the queue blocks put when full and take when empty and does the locking internally:

```
BlockingQueue<Integer> counter = new ArrayBlockingQueue<>(5);
```

```
cook: for (int t = 1; t <= 20; t++) counter.put(t);          students: collected[id] += counter.take();   // output: Sum ... 210
```

Part A with tryLock(timeout) (Wallet.transferTryLock): lock from with tryLock(5 ms); if the second lock is not obtained within 5 ms, **release the first**, wait a random 1–10 ms and retry. Deadlock cannot persist because the *hold-and-wait* condition is broken (a thread gives up what it holds).

```
while (true) {
   if (from.lock.tryLock(5, MILLISECONDS)) {
      try { if (to.lock.tryLock(5, MILLISECONDS)) { try { move money; return; } finally { to.lock.unlock(); } } }
      finally { from.lock.unlock(); }              // could not get the second lock: give up the first
   }
   Thread.sleep(random 1..10 ms);                  // random back-off avoids live-lock
}
```

N tokens (1 producer, 2 consumers, capacity 5)	**wait()/notifyAll() (best of 3)**	**ArrayBlockingQueue(5) (best of 3)**
10000	16 ms (1.56 µs/token)	24 ms (2.41 µs/token)
50000	54 ms (1.08 µs/token)	58 ms (1.17 µs/token)
100000	103 ms (1.03 µs/token)	105 ms (1.05 µs/token)

Transfers (8 threads × 1,000 random, 6 wallets, no pause)	**Time**	**Per transfer**
ordered synchronized locks	32 ms	4.76 µs
tryLock(5 ms) + back-off	317 ms	47.48 µs

Space	**wait/notify counter**	**ArrayBlockingQueue(5)**

bytes per object (measured)	130 (int[5] + fields + monitoring list; the counter itself is 5 ints + 3 ints)	188 (Object[5] + ReentrantLock + 2 Conditions; boxed Integers above 127 add 16 bytes each)

Analysis. put/take are **O(1)** in both implementations; measured 1.0–1.6 µs per token for wait/notifyAll and 1.0–2.4 µs for the queue – equal for practical purposes (the cost is the thread hand-over, not the data structure). Space is O(capacity) for both (a few hundred bytes). **Transfers:** ordered locking costs one acquisition per wallet, Θ(1) (≈ 4–5 µs here); tryLock is about **10× slower** under contention (≈ 47 µs per transfer) because failed attempts wait up to 5 ms and then back off, and on one core a thread holding a lock is often descheduled, so the others time out and retry. **Which design is less error-prone?** ArrayBlockingQueue – the while/notifyAll protocol, whose three classic mistakes (if, notify, missing synchronized) all failed in the experiments above, simply does not exist. For transfers, **lock ordering** is simpler and faster, but it needs a global ordering key that every caller respects (easy to violate when a third lock appears); tryLock tolerates a violated order but adds timeouts, retries and possible live-lock, so it is the better choice only when no ordering can be defined.

Q7 – Design a Concurrent, Fault-Tolerant Processor from First Principles (10 marks, K5)

Stage 1 – Interpret the problem (joint, 2 marks)

1.1 Tasks that can run in parallel, and the data that is shared

	Content
Independent tasks (can run in parallel)	"Read, validate and score ONE file" – the 10,000 tasks do not depend on each other: each reads its own file, parses its own text and computes its own result. This is an I/O-bound job (waiting for the storage), so overlapping tasks shortens the total time.
Data that is NOT shared	File content, the parsing state and the score of one student – private to one task (on its stack).
Data that IS shared	(1) the **result collection** that every task writes to and the summary reads; (2) the list of files (read-only after creation, so safe); (3) the worker threads' queue (managed by the executor).
Must be sequential	Building the summary – it needs all results, so it must happen only after every task has finished.

1.2 Every failure that can occur and how each must be handled

Failure	**Exception raised**	**Handling**	**Recorde**
File listed but missing	NoSuchFileException	Caught in the task	MISSING
Empty file; wrong number of fields; bad student id; mark not a	CorruptFileException (user-defined, checked)	Caught in the task	CORRUI reason

number; mark outside 0–100			
Bytes that are not valid text (binary garbage)	CharacterCodingException	Caught in the task (found by testing, see Stage 3)	CORRUI
Path is a directory / unreadable / I/O error	IOException	Caught in the task	ERROR message
Programming bug inside a task	RuntimeException	Caught in the task; a second safety net in the processor (ExecutionException from Future.get())	ERROR
An Error thrown inside a task	e.g. AssertionError (injected)	Not caught by the task; the Future captures it and the processor records it	ERROR
Interrupt during the run	InterruptedException	Flag restored, task ends	ERROR ("interrup
Pool cannot be stopped in time	–	shutdown() + awaitTermination + shutdownNow() in finally	–

1.3 Requirements stated as measurable statements

#	**Requirement**	**How it is measured**
R1	A failure in one file must never stop or fail the run	Fault injection: with bad files **and** exceptions thrown inside tasks the run

		still finishes and reports all files
R2	Completeness: every submitted file appears in the summary **exactly once**	success + corrupt + missing + error == submitted (the complete flag) in every run
R3	Correctness: summary counts equal the true counts	Hand-checked set of 15 files; generated set of 10,000 files compared with the ground truth written by the generator
R4	Each failed file is listed with its name and reason	Failure list of the summary compared with the injected faults
R5	Speed: 10,000 files must finish in a time that falls as threads increase while the work is I/O-bound	Total time with pools of 1, 2, 4, 8 (and 16, 32) threads, 7 runs each
R6	Resources: at most p worker threads; no file handle left open	Peak thread count; every file read through Files.readString (closed by the call)
R7	The result must not depend on thread count or scheduling	Same counts for every pool size and every run (correct = true in all 54 timed runs)

Stage 2 – Design (Student A, 3 marks)

2.1 Design with ExecutorService, Callable and Future

- ResultProcessor.process(files, p, latency) creates **Executors.newFixedThreadPool(p)** – p worker threads re-used for all 10,000 tasks.
- One **Callable<FileResult>** (ScoreFileTask) is submitted per file; submit returns a **Future<FileResult>** that is kept in a list.
- A task **never throws for an expected problem**: it converts every failure into a FileResult with a Status, writes it into the shared map and returns it.
- The processor then calls future.get() on every future – this **waits** for the task and is the safety net: if the task itself threw (bug or Error), get() throws ExecutionException and the processor records an ERROR result for that file.
- Only after all futures are done and the pool is shut down (in finally) is the **Summary** built from the map – so the summary can never see a half-finished run.

2.2 Choice of the concurrent collection for the shared results, and justification

ConcurrentHashMap<String, FileResult> keyed by file name.

Candidate	**Verdict**
HashMap / ArrayList (not thread-safe)	**Rejected** – concurrent put/add can lose entries or corrupt the structure (the same race as Q5).
Collections.synchronizedList/Map	Correct, but **one lock** for all 10,000 writers: threads queue up on it.
CopyOnWriteArrayList	**Rejected** – every add copies the whole array: O(n) per write, O(n^2) for 10,000 results; designed for read-mostly data.
ConcurrentLinkedQueue	Fast and non-blocking, but cannot answer "is this file already recorded?" and has no key – the summary could not detect a missing or duplicate entry.

ConcurrentHashMap (chosen)	Many writers at once with fine-grained locking/CAS (low contention); put and putIfAbsent are atomic, O(1) expected; the **key = file name** makes it impossible to record a file twice and lets putIfAbsent act as the "record only if the task did not" operation used in the ExecutionException fallback; size() after the run is the completeness check.

2.3 Exception handling so that one failure does not stop the run

- **Layer 1 – inside the task:** catch blocks ordered from narrow to broad (NoSuchFileException → MISSING, CorruptFileException → CORRUPT, CharacterCodingException → CORRUPT, IOException → ERROR, InterruptedException → ERROR with the interrupt flag restored, RuntimeException → ERROR). The sink.put(...) is executed on every path, so exactly one result exists per file.
- **Layer 2 – in the processor:** Future.get() is wrapped in try/catch (ExecutionException | CancellationException); the file gets an ERROR result via putIfAbsent. This also covers an Error (e.g. AssertionError) that the task did not catch.
- **Layer 3 – resources:** pool.shutdown() in finally, awaitTermination, then shutdownNow(); files are read with Files.readString, which opens and closes the file itself (nothing to leak).
- Why the run cannot be stopped: no exception is allowed to leave the loop over the futures, and no task depends on another.

2.4 Class diagram

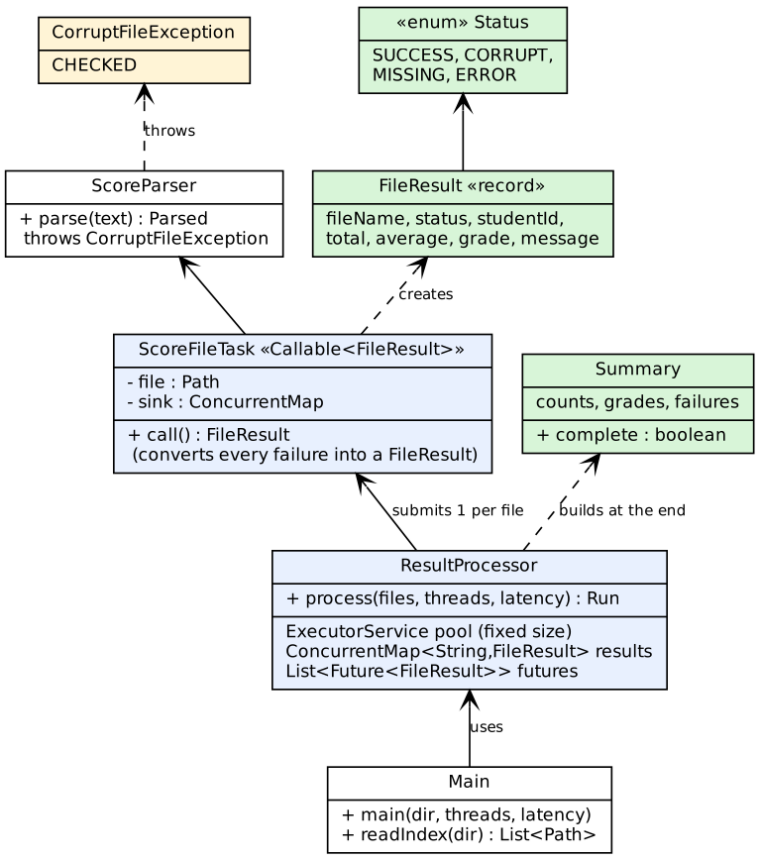


Figure Q7.1 – Class diagram of the result processor

2.5 Pseudocode

```
ALGORITHM ProcessAll
INPUT  : files[1..N], p (pool size), latency (simulated, may be 0)
OUTPUT : Summary
BEGIN
   results ← NEW ConcurrentHashMap()              // shared, thread-safe
   pool    ← NEW FixedThreadPool(p)
   futures ← EMPTY LIST
   TRY
      FOR EACH file IN files DO
         futures.ADD( pool.SUBMIT( NEW ScoreFileTask(file, results, latency) ) )
      END FOR
      FOR EACH (file, future) IN futures DO       // wait for every task
         TRY    future.GET()
         CATCH ExecutionException, CancellationException
            results.PUT_IF_ABSENT(name(file), FileResult(name(file), ERROR, ...))   // safety net
         END TRY
      END FOR
   FINALLY
```

```
    pool.SHUTDOWN();                IF      NOT      pool.AWAIT_TERMINATION(30      s)      THEN
pool.SHUTDOWN_NOW()
  END TRY
  RETURN Summary(N, results.VALUES())            // built only after everything has finished
END

ALGORITHM ScoreFileTask.Call(file)                  // runs on a worker thread
BEGIN
  TRY
    IF latency > 0 THEN SLEEP(latency)            // simulated slow storage
    text   ← READ_WHOLE_FILE(file)
    parsed ← ParseScore(text)                     // may raise CorruptFileException
    result ← FileResult(SUCCESS, id, total, average, grade)
  CATCH NoSuchFile          : result ← FileResult(MISSING)
  CATCH CorruptFile         : result ← FileResult(CORRUPT, reason)
  CATCH CharacterCoding     : result ← FileResult(CORRUPT, "not valid text")
  CATCH IOException         : result ← FileResult(ERROR, message)
  CATCH Interrupted         : RESTORE interrupt flag;  result ← FileResult(ERROR, "interrupted")
  CATCH RuntimeException    : result ← FileResult(ERROR, class + message)
  END TRY
  results.PUT(name(file), result)                 // EVERY path records exactly one result
  RETURN result
END

ALGORITHM ParseScore(text)
  IF text IS BLANK THEN RAISE CorruptFile("empty file")
  fields ← SPLIT(text, ",");  IF COUNT(fields) ≠ 6 THEN RAISE CorruptFile(...)
  IF fields[1] DOES NOT MATCH "S" + 5 digits THEN RAISE CorruptFile(...)
  FOR i ← 2 TO 6 DO  m ← INTEGER(fields[i]) OR RAISE CorruptFile;  IF m < 0 OR m > 100 THEN
RAISE CorruptFile
  total ← SUM(marks);  average ← total / 5
  grade ← FAIL IF any mark < 35;  ELSE A (≥75), B (≥60), C (≥50), D (≥40), otherwise FAIL
```

```
package results;

import java.io.IOException;
import java.nio.file.*;
import java.util.concurrent.Callable;
import java.util.concurrent.ConcurrentMap;
```

```
/** One unit of work: read, validate and score ONE file. It converts every expected failure into a FileResult.
*/
public class ScoreFileTask implements Callable<FileResult> {
   private final Path file; private final ConcurrentMap<String, FileResult> sink; private final long latencyMicros;

   public ScoreFileTask(Path file, ConcurrentMap<String, FileResult> sink, long latencyMicros) {
      this.file = file; this.sink = sink; this.latencyMicros = latencyMicros;
   }

   @Override public FileResult call() {
      String name = file.getFileName().toString();
      FileResult r;
      try {
         if (latencyMicros > 0) Thread.sleep(latencyMicros / 1000, (int) (latencyMicros % 1000) * 1000);   // SIMULATED storage latency
         String content = Files.readString(file);                    // NoSuchFileException if missing
         ScoreParser.Parsed p = ScoreParser.parse(content);
         r = FileResult.ok(name, p.id(), p.total(), p.average(), p.grade());
      } catch (NoSuchFileException e) {
         r = FileResult.failed(name, Status.MISSING, "file not found");
      } catch (CorruptFileException e) {
         r = FileResult.failed(name, Status.CORRUPT, e.getMessage());
      } catch (java.nio.charset.CharacterCodingException e) {          // FIX after the hand-checked test: bytes that are not valid text
         r = FileResult.failed(name, Status.CORRUPT, "not valid UTF-8 text");
      } catch (IOException e) {
         r = FileResult.failed(name, Status.ERROR, "I/O error: " + e.getMessage());
      } catch (InterruptedException e) {
         Thread.currentThread().interrupt();                         // keep the interrupt flag for the pool
         r = FileResult.failed(name, Status.ERROR, "interrupted");
      } catch (RuntimeException e) {                                 // a bug must not stop the run either
         r = FileResult.failed(name, Status.ERROR, "unexpected " + e.getClass().getSimpleName() + ": " + e.getMessage());
      }
      sink.put(name, r);                                  // shared result map: many threads write concurrently
      return r;
   }
}
```

```
package results;

import java.nio.file.Path;
import java.util.*;
import java.util.concurrent.*;

/** Design: ExecutorService + Callable + Future + ConcurrentHashMap. One bad file can never stop the run.
*/
public class ResultProcessor {
   public record Run(Summary summary, double millis) { }

   /** Creates the Callable for one file (a test can supply a faulty one). */
   public interface TaskFactory { Callable<FileResult> create(Path file, ConcurrentMap<String, FileResult>
sink, long latencyMicros); }

   public static Run process(List<Path> files, int threads, long latencyMicros) throws InterruptedException {
      return process(files, threads, latencyMicros, ScoreFileTask::new);
   }

   public static Run process(List<Path> files, int threads, long latencyMicros, TaskFactory factory) throws
InterruptedException {
      ConcurrentMap<String, FileResult> results = new ConcurrentHashMap<>(Math.max(16, files.size() *
2));
      ExecutorService pool = Executors.newFixedThreadPool(threads);
      long t0 = System.nanoTime();
      List<Map.Entry<Path, Future<FileResult>>> futures = new ArrayList<>(files.size());
      try {
         for (Path f : files) futures.add(Map.entry(f, pool.submit(factory.create(f, results, latencyMicros))));
         for (Map.Entry<Path, Future<FileResult>> e : futures) {
            String name = e.getKey().getFileName().toString();
            try {
               e.getValue().get();                          // wait for this file's task
            } catch (ExecutionException ex) {                    // the task itself threw: still record the file
               results.putIfAbsent(name, FileResult.failed(name, Status.ERROR, "task failed: " +
ex.getCause()));
            } catch (CancellationException ex) {
               results.putIfAbsent(name, FileResult.failed(name, Status.ERROR, "cancelled"));
            }
         }
      } finally {
```

```
            pool.shutdown();                                    // always release the worker threads
            if (!pool.awaitTermination(30, TimeUnit.SECONDS)) pool.shutdownNow();
        }
        double ms = (System.nanoTime() - t0) / 1e6;
        return new Run(new Summary(files.size(), results.values()), ms);
    }
}
```

Stage 3 – Implement and measure (Student B, 3 marks)

3.1 Implementation, with corrupt and missing files injected

GenerateScores creates N files with a fixed seed: 2 % are **missing** (listed in index.txt but never created), 3 % **corrupt** in three ways (non-numeric mark, mark 150, empty file), the rest valid. It also writes expected_summary.txt – the ground truth. The 15-file hand-checked set Q7/input/handset adds the unusual faults (directory instead of file, binary bytes, wrong field count).

3.2 Summary counts verified for a small hand-checked set (15 files)

File	**Content**	**Hand-calculated result**
f01	S00001,80,70,90,60,75	SUCCESS: total 375, average 75.0, grade **A**
f02	S00002,60,65,70,55,62	SUCCESS: 312, 62.4, **B**
f03	S00003,50,52,55,51,53	SUCCESS: 261, 52.2, **C**
f04	S00004,45,42,40,44,41	SUCCESS: 212, 42.4, **D**
f05	S00005,90,90,90,90,30	SUCCESS: 390, 78.0, **FAIL** (one

		mark below 35)
f06	S00006,100,100,100,100,100	SUCCESS: 500, 100.0, **A**
f07	S00007,0,0,0,0,0	SUCCESS: 0, 0.0, **FAIL**
f08	S00008,70,eighty,60,50,40	CORRUPT (mark not a number)
f09	S00009,70,60,50,40,101	CORRUPT (mark out of range)
f10	(empty file)	CORRUPT (empty)
f11	S00011,70,60,50	CORRUPT (4 fields instead of 6)
f12, f13	(files do not exist)	MISSING ×2
f14	bytes FF FE 00 80 C3 28	CORRUPT (not valid text)
f15	a directory named f15.txt	ERROR (cannot be read as a file)

Hand totals: **7 successes, 5 corrupt, 2 missing, 1 error**; grades A = 2 (f01, f06), B = 1, C = 1, D = 1, FAIL = 2 (f05, f07); mean of the seven averages = (75 + 62.4 + 52.2 + 42.4 + 78 + 100 + 0) / 7 = **58.57**.

```
$ java -cp out results.Main input/handset 4
SUMMARY  submitted=15 success=7 corrupt=5 missing=2 error=1   complete=true
grades   {A=2, B=1, C=1, D=1, FAIL=2}   mean average of successful files = 58.57
failures (8):
```

```
  f08.txt [CORRUPT] mark 2 is not a number: 'eighty'
  f09.txt [CORRUPT] mark 5 out of range 0..100: 101
  f10.txt [CORRUPT] empty file
  f11.txt [CORRUPT] expected 6 fields, found 4
  f12.txt [MISSING] file not found
  f13.txt [MISSING] file not found
  f14.txt [CORRUPT] not valid UTF-8 text
  f15.txt [ERROR] I/O error: Is a directory
time = 37 ms with 4 thread(s), simulated latency 0 us
expected  submitted=15 success=7 corrupt=5 missing=2 error=1
actual    submitted=15 success=7 corrupt=5 missing=2 error=1
MATCH
```

The program output equals the hand calculation exactly (MATCH). For the large set, the same check against the generator's ground truth:

```
$ java -cp out results.GenerateScores 10000 data; java -cp out results.Main data 4
generated 10000 entries in data: submitted=10000 success=9548 corrupt=260 missing=192 error=0
SUMMARY  submitted=10000 success=9548 corrupt=260 missing=192 error=0   complete=true
grades   {A=1291, B=3952, C=1201, D=119, FAIL=2985}   mean average of successful files = 64.79
failures (452):
  ... 442 more
time = 460 ms with 4 thread(s), simulated latency 0 us
expected  submitted=10000 success=9548 corrupt=260 missing=192 error=0
actual    submitted=10000 success=9548 corrupt=260 missing=192 error=0
MATCH
```

3.3 A failure that was not handled correctly (recorded as it happened)

In my **first version** of ScoreFileTask there was no handler for CharacterCodingException. The hand-checked set exposed it at once: the binary file f14.txt was reported as **ERROR ("I/O error: Input length = 1")** instead of **CORRUPT**, so the summary read corrupt=4 error=2 instead of corrupt=5 error=1 (MISMATCH). The cause: Files.readString decodes UTF-8 and throws MalformedInputException (a subclass of IOException) for invalid bytes; it fell into the generic IOException handler. Fix: a catch (CharacterCodingException e) placed before catch (IOException e) → CORRUPT. The failing run is preserved in Q7/output/first_run_defect.txt:

```
$ java -cp out results.Main input/handset 4          # FIRST version of ScoreFileTask (no CharacterCodingException handler)
  f11.txt [CORRUPT] expected 6 fields, found 4
  f14.txt [ERROR] I/O error: Input length = 1
expected  submitted=15 success=7 corrupt=5 missing=2 error=1
```

```
actual    submitted=15 success=7 corrupt=4 missing=2 error=2
MISMATCH
```

Fault injection inside the tasks (FaultInjectionTest): the task for f03 (a good file) throws an IllegalStateException and the task for f08 throws an AssertionError (an Error). The run still completed, and both files were recorded as ERROR by the Future safety net:

```
expected: 15 submitted; f03 (was SUCCESS) and f08 (was CORRUPT) now ERROR -> success=6 corrupt=4
missing=2 error=3 (1 directory + 2 injected)
actual  : submitted=15 success=6 corrupt=4 missing=2 error=3
PASS - the run was not stopped, every file is accounted for, and the two injected failures are reported
```

(My first expected value in this test contained an arithmetic slip – error should be 1 + 2 = 3, not 4. The program's answer was right and the expectation in the test was corrected.)

3.4 Total time with pools of 1, 2, 4 and 8 threads (10,000 files)

Two settings were measured, because they answer different questions. **A:** real file reading only. **B:** the same plus a **simulated 500 µs storage latency per file** (a network drive or busy disk) – this is a *simulation*, stated as such. 25 warm-up runs preceded the timed runs; each configuration was timed 7 times (A) or 2 times (B); raw data: Q7/output/measurements.csv. The machine has **one** CPU core.

Threads	**A: all runs (ms)**	**best**	**median**	**speed-up**	**s c**
1	88, 153, 131, 96, 90, 95, 88	88	95	1.00×	t
2	45, 50, 43, 44, 38,	36	43	2.21×	t

	38, 36				
4	35, 41, 37, 36, 36, 42, 48	35	37	2.57×	tr
8	40, 38, 37, 41, 47, 43, 40	37	40	2.37×	tr
16	44, 42, 39, 42, 43, 44, 41	39	42	2.26×	tr
32	46, 58, 55, 53, 46, 46, 49	46	49	1.94×	tr

Threads	**B: all runs (ms)**	**best**	**median**	**speed-up**
1	6368, 6354	6354	6368	1.00×

2	3204, 3175	3175	3204	1.99×
4	1572, 1572	1572	1572	4.05×
8	784, 804	784	804	7.92×
16	404, 406	404	406	15.67×
32	189, 200	189	200	31.84×

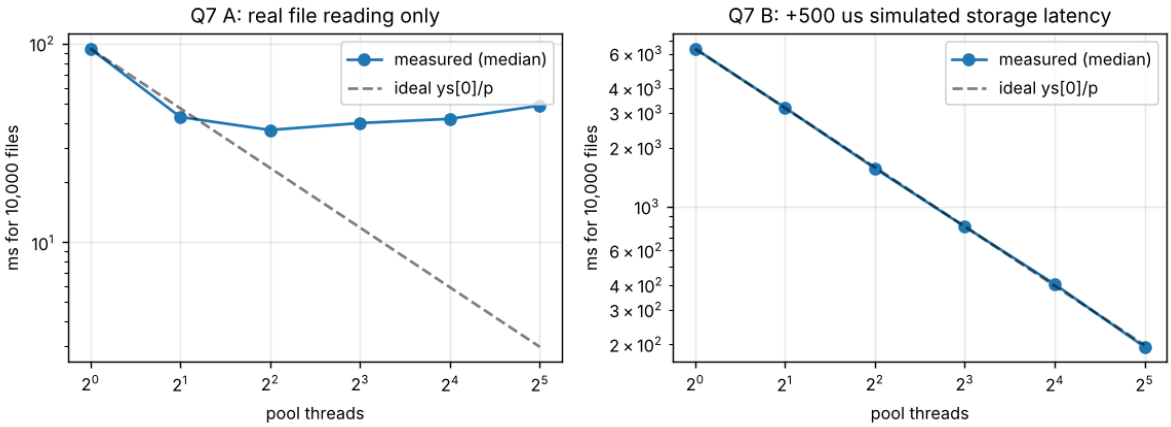


Figure Q7.2 – Total time for 10,000 files vs pool size (log–log, dashed = ideal speed-up)

Stage 4 – Defend the design (joint, 2 marks)

4.1 Recommended pool size, and why more threads did not always help

Recommendation: make the pool size a parameter; use 8 as the default, and about 32 for slow network storage – but find the exact value by measuring on the real storage. Evidence: (A) with fast local reads the time fell from 95 ms (1 thread) to 37 ms (4 threads, 2.6×) and then **rose again** – 40 ms at 8, 42 ms at 16, 49 ms at 32 – so more than about 4–8 threads brought nothing; (B) with 500 µs of waiting per file the speed-up was almost perfectly linear: 2.0× (2), 4.1× (4), 7.9× (8), 15.7× (16), 31.8× (32), because a waiting thread uses no CPU. A useful rule: **threads ≈ cores × (1 + wait time / compute time)**. Here compute ≈ 4 µs per file and wait 500 µs, so the rule allows far more than 8 threads for slow storage, while for fast local reads wait ≈ 0 and the rule gives ≈ the number of cores.

Why more threads did not always help: (1) there is nothing left to overlap once the CPU or the disk is saturated – this machine has one core, so extra threads can only wait; (2) every extra thread adds context switches, scheduler work and contention on the shared queue and map (A: 49 ms at 32 threads against 37 ms at

4); (3) each thread costs memory (about 1 MB of reserved stack and ≈ 76 KB resident, Q4); (4) the storage device has its own limit on concurrent requests. Because the sandbox has one core, the 2×–2.6× gain in A comes from overlapping blocking system calls and the submission work, not from parallel computation – on a multi-core computer the numbers will differ and must be re-measured.

4.2 How the design guarantees a complete and correct summary

1. **Exactly one result per file:** every path through ScoreFileTask.call() – success, each catch, the generic RuntimeException catch – ends in sink.put(fileName, result); the key is the file name, so a file cannot be recorded twice.
2. **Safety net:** if a task fails in a way the task itself did not handle (a bug, an Error), Future.get() throws ExecutionException and the processor records an ERROR result with putIfAbsent – tested by fault injection.
3. **The summary is built only after all tasks have finished:** the processor calls get() on every future (and shuts the pool down) before creating Summary, and Future.get() establishes a happens-before relation, so the main thread sees every result.
4. **Self-check:** Summary.complete is true only if success + corrupt + missing + error == submitted; it was true in all 54 timed runs, the fault-injection run and the correctness runs.
5. **Correctness:** the counts equal the hand calculation (15 files) and the generator's ground truth (10,000 files), for every pool size.

4.3 One remaining limitation

There is **no time limit for a single file**. A read that blocks forever (a network share that stops responding) occupies a worker thread permanently; if all p workers hang, the run never completes (future.get() has no timeout). A production version would use future.get(timeout) with cancel(true) and record such files as ERROR ("timed out"). Two lesser limitations: all results are held in memory (10,000 small records is fine, millions would need streaming), and duplicate file names in the index would collide on the map key – the complete check would flag that run as incomplete instead of silently succeeding.

Alternative Concept and Complexity Analysis – parallelStream() or one thread per file

Parallel streams: files.parallelStream().forEach(f -> new ScoreFileTask(f, sink, lat).call()) – no executor code, the work runs on the JVM-wide **common ForkJoinPool**, whose parallelism is about *cores − 1* (here **1**), not a number we choose, and it is meant for CPU-bound work: blocking I/O inside it starves other parallel streams.
Thread per file: new Thread(task).start() for each of the 10,000 files.

```
files.parallelStream().forEach(f -> new ScoreFileTask(f, sink, lat).call());      // alternative 1
for (Path f : files) { Thread t = new Thread(new FutureTask<>(new ScoreFileTask(f, sink, lat))); t.start(); }   // alternative 2
```

Approach (10,000 files)	**Total time**	**Threads and memory**	**Failure of one file**
ExecutorService, p = 8, 500 µs latency	804 ms (p = 32: 200 ms)	p threads: 8 × ≈ 76 KB resident (8 × 1 MB reserved)	Handled: one result per file; run continues (R1)
parallelStream(), no latency	317 ms	common pool: 1 worker here (cores − 1) + caller; O(cores) threads	Handled **only because the task converts failures**; see next row
parallelStream(), 500 µs latency	3,410 ms (executor with 8 threads: 804 ms)	same – cannot use more than the common pool's parallelism; blocking waits waste the few workers	same
parallelStream(), **no** per-element handling	aborted after **89 of 10,000** files, in 55 ms	same	**The first corrupt file aborted the whole run**; the other results are lost and no summary exists
One thread per file, 500 µs latency	1,768 ms	peak 55 live threads (they finish quickly);	Handled (same task), plus the cost below

		resident +39.8 MB	
One thread per file, slow storage (2 s per file)	7,334 ms	**peak 10,006 live threads**; resident **+267,812 KB (≈ 262 MB)**; reserved virtual **+10,243,292 KB (≈ 9.8 GB)** = 10,006 × 1 MB stack	Handled, but the process nearly exhausted memory/thread limits

```
$ java -cp out results.Alternatives parstream-naive data 0
mode = parstream-naive, files = 10000, simulated latency = 0 us, processors = 1, common pool parallelism = 1
whole run ABORTED after 89 of 10000 files were processed: java.lang.RuntimeException: score02521.txt: results.CorruptFileException: empty file
no summary can be produced; the other files' results are lost (not collected anywhere).
time until abort = 55 ms
```

Analysis. *Total time:* the executor needs about ⌈N/p⌉ × (wait + compute) = Θ(N/p) for p up to the useful limit (measured 804 ms at p = 8 for N = 10,000, 500 µs); parallel streams are limited by the common pool's parallelism *c* (≈ cores − 1, here 1), so Θ(N/c) – 3,410 ms; one thread per file is Θ(N/min(N, thread limit)) in theory, but only as fast as threads can be created (≈ 1.8 s for 10,000 here: ≈ 177 µs of creation and scheduling per file, while the pool needed 0.6 µs per task in Q4). *Memory = threads × stack size:* the executor: p × 1 MB = **8 MB** reserved; the common pool: c × 1 MB; one thread per file: **N × 1 MB = about 9.8 GB reserved** (measured 10.2 GB) and ≈ 262 MB resident – O(N) memory instead of O(p), and the JVM can fail with OutOfMemoryError: unable to create native thread on a computer with less memory or a lower process limit. *Failure handling:* in a parallel stream an exception that escapes the lambda **terminates the whole pipeline** (89 files processed, no summary), so it is correct only if every element handles its own failures – the same discipline as the task of the executor design; with raw threads an uncaught exception ends only that thread, and its result is silently missing unless the task records it. **Conclusion:** the fixed pool is retained: bounded threads and memory, a tunable degree of parallelism for I/O-bound work, explicit Future-based failure handling and a guaranteed complete summary.

Conclusion

Exceptions were classified, ordered and chained so that no failure information is lost (Q1, Q2); resource safety was demonstrated with breakpoints, counted handles and try-with-resources (Q3); thread behaviour was observed, not assumed, over 24 recorded runs (Q4); the race condition, the deadlock and the producer–consumer defects were each reproduced, explained and fixed or prevented (Q5, Q6); and a fault-tolerant concurrent processor was designed from requirements, implemented, tested with injected faults and defended with measurements (Q7). Wherever an experiment could not show an effect on the single-core test machine, that is stated and the way it was made visible is described. The alternative concepts were implemented or measured, and in each case the chosen design was retained for a stated, evidence-based reason.